

Università degli Studi di Torino

Dipartimento di Informatica



Tesi di Laurea Triennale in Informatica

**ZapGate: Progettazione e valutazione di
un modello per micropagamenti
verificabili su Nostr tramite LN**

Relatore:
Prof. Andrea Bracciali

Candidato:
Giordano Davide

Anno Accademico 2024/2025

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Sommario

Questa tesi studia come rendere possibili **micropagamenti sicuri e verificabili** all'interno dell'ecosistema **Nostr**, con l'obiettivo di garantire all'utente maggiore libertà e una buona **esperienza d'uso**. In altre parole, si affronta, a partire dal citato Nostr, questo nuovo protocollo decentralizzato e resistente alla censura, il problema di come rendere i pagamenti molto piccoli rapidi, semplici e affidabili, senza dover passare da una piattaforma centrale.

Il lavoro è stato sviluppato in tre fasi principali. Prima è stato analizzato il funzionamento di Nostr e del sistema di micropagamenti chiamato *zap*, valutandone i limiti sia in scenari dove l'utente affida i propri fondi a terzi (*custodial*) sia in scenari in cui mantiene il pieno controllo (*non-custodial*). Poi è stato realizzato un prototipo, chiamato **ZapGate**, che mostra come sia possibile inviare e ricevere micropagamenti integrati con Nostr. Infine, è stato costruito un **modello di business** per stimarne l'impatto economico e le opportunità di mercato.

I risultati principali mostrano che:

- i micropagamenti possono essere eseguiti in modo sicuro e senza intermediari, lasciando all'utente il pieno controllo dei propri fondi;
- il prototipo realizzato funziona con diversi client e portafogli digitali, garantendo compatibilità e interoperabilità;
- l'integrazione dei micropagamenti apre nuove possibilità di sostenibilità per piattaforme e sviluppatori.

La tesi si conclude con linee guida per l'uso pratico di ZapGate e con alcune proposte di sviluppo futuro, come sistemi di delega più flessibili, meccanismi di reputazione e strumenti per collegare meglio i diversi relay della rete.

Indice

I	Tecnologia e architettura del protocollo	1
1	Introduzione	2
1.1	Critiche ai social centralizzati	2
1.2	Necessità di decentralizzazione	2
2	Il protocollo Nostr	4
2.1	Relay e client	4
2.2	Identità, chiavi e crittografia	4
2.3	Outbox e inbox	5
2.4	Struttura degli eventi e interoperabilità	5
2.5	WebSocket, subscription e filtri	5
2.6	Estensioni del protocollo (NIP)	6
2.6.1	Criteri di accettazione	6
3	Estensioni del protocollo e micropagamenti	8
3.1	Bitcoin e limiti on-chain	8
3.2	Canali, fatture BOLT11 e schema LNURL	9
4	Lightning Network (LN) e Zap	11
4.1	Attori coinvolti	11
4.2	Sicurezza, autenticità e integrità	12
4.3	Proprietà garantite (ipotesi forti)	12
4.4	Analisi in condizioni forti	13
5	Simulazione tecnica	15
5.1	Scenario non-custodial: Polar	15
5.1.1	Ambiente	15
5.1.2	Setup, funding e apertura del canale	16
5.2	Scenario custodial: LNbits e Zeus Embedded LND	19
5.3	Confronto tra modelli	19
5.4	Mini-valutazione quantitativa	20

6	Analisi delle vulnerabilità	23
6.1	Attacchi possibili anche con ipotesi forti	23
6.2	Attacchi che richiedono il venir meno delle ipotesi forti	24
6.3	Contromisure tecniche	25
6.3.1	Reperimento dell'indirizzo LNURL	25
6.3.2	Emissione della ZapRequest	25
6.3.3	Pagamento via LNURL	25
6.3.4	Emissione della ZapReceipt	26
6.3.5	Visibilità tra relay	26
6.4	Rischi chiave e mitigazioni	26
6.5	Considerazioni di sicurezza	27
II	Applicazioni e Modelli di Business	28
7	Applicazioni, Modelli di Business e ZapGate	29
7.1	Zap come abilitante per nuovi modelli economici	29
7.1.1	Integrazione nei client	30
7.1.2	Rimozione di intermediari	30
8	ZapGate: applicazione concreta	32
8.1	Cos'è una "killer app"	32
8.2	Perché ZapGate? Obiettivi e funzionalità	32
8.3	Architettura generale	33
8.4	Testing del prototipo	35
8.5	Implementazione tecnica	36
8.6	Sicurezza e configurazione	36
8.7	Limitazioni del prototipo	37
8.8	Validità della demo come proof-of-concept	38
9	Analisi economica del modello Zap nell'editoria digitale: il caso ZapGate	39
9.1	Lightning Network come abilitatore dei micropagamenti . . .	39
9.2	Simulazione su larga scala	40
9.3	Il ruolo del provider	41
9.4	Proiezioni e impatto	41
9.5	Applicazione all'editoria digitale	42
9.6	Modelli di monetizzazione	43
9.7	Scenari di applicazione	43
9.8	Complementarità con il modello ad abbonamento	45
9.9	Distribuzione dei ricavi e legge di Pareto	46
9.10	Punti di forza e criticità del modello Zap	47
9.11	Prospettive e lavori correlati	48

III	Considerazioni Finali	50
10	Conclusioni	51
IV	Bibliografia e Appendici	54
	Appendice A – Setup e Replicabilità	58
	Appendice B – Ulteriori Materiali	60
	Appendice C – Formati dei messaggi Nostr	61

Parte I

Tecnologia e architettura del protocollo

1 Introduzione

1.1 Critiche ai social centralizzati

Negli ultimi anni, tantissime tra le principali piattaforme social hanno ricevuto critiche per il loro modello centralizzato. Non parliamo di outsider, ma di veri e propri colossi come Meta, X (ex Twitter) e TikTok, che controllano molto l'accesso ai contenuti, le regole di moderazione, la visibilità dei post e l'identità degli utenti, peraltro in maniera esplicita. Questo approccio consente efficienza e scalabilità da una parte, ma solleva d'altro canto problemi non indifferenti sotto numerosi aspetti:

- **Censura arbitraria:** post o account possono essere rimossi o oscurati senza processo trasparente;
- **Proprietà dei dati:** gli utenti non possiedono realmente contenuti e identità digitale;
- **Lock-in e sorveglianza:** i dati vengono monetizzati e profilati a fini pubblicitari, con impatto sulla privacy;
- **Dipendenza da singoli attori:** l'infrastruttura può essere manipolata da chi la gestisce, limitando la libertà di espressione.

Tali criticità emergono non solo nell'esperienza quotidiana di milioni di utenti, ma anche in settori a propensione più politica e giornalistica. Il problema è quindi tecnico, etico e sistemico.

1.2 Necessità di decentralizzazione

Proprio in contrapposizione a questo modello, negli ultimi anni si sono sviluppati sistemi alternativi fondati su logiche **decentralizzate**. Cerchiamo di spiegare meglio: quando diciamo che un sistema è decentralizzato intendiamo che non c'è un unico “padrone” o server che controlla tutto, ma tanti nodi indipendenti che collaborano tra loro. In questo modo i dati non stanno in un solo posto, ma risultano distribuiti: se un nodo si blocca o viene spento, gli altri continuano a funzionare.

Decentralizzato sì, ma non lasciato a se stesso: la comunicazione tra nodi avviene seguendo regole comuni, ad esempio sono il fatto che ognuno può controllare l'autenticità delle informazioni grazie alla crittografia e decidere quali dati conservare o condividere. Questo rende il sistema più resistente a censure o guasti e dà ad ogni utente più libertà e controllo sulla propria identità e sui propri contenuti. La decentralizzazione, dunque, distribuisce il controllo, evitando punti singoli di fallimento o di censura. In questo paradigma:

- ogni utente può accedere e pubblicare contenuti senza dover passare da una piattaforma centrale;
- l'identità dell'utente è sua e può portarla ovunque, senza dipendere da un singolo servizio.

In questo scenario si inserisce **Nostr** (*Notes and Other Stuff Transmitted by Relays*), un protocollo aperto per la comunicazione libera, interoperabile e resistente alla censura, che vedremo con maggiore attenzione e dettaglio nel capitolo seguente.

2 Il protocollo Nostr

Come prima cosa, è necessario fare un po' di chiarezza sugli strumenti con cui lavoreremo: Nostr non è un'applicazione, bensì un protocollo di comunicazione. È dunque un insieme di regole e standard che consentono a client diversi di interagire tra loro in modo interoperabile, utilizzando eventi pubblici e feed personali come veicoli di informazione. All'interno del protocollo, gli attori fondamentali sono due: *relay* e *client*.

2.1 Relay e client

La rete si basa su relay (server indipendenti, di norma raggiungibili via `wss://`) che ricevono, conservano e diffondono i messaggi senza modificarli. Ogni relay è gestito autonomamente e può stabilire proprie regole di accesso e pubblicazione. Non comunicando tra loro in modo obbligatorio e non essendoci un ente centrale di coordinamento, il sistema risulta resistente a blocchi e manipolazioni.

Il client è invece l'interfaccia con cui l'utente interagisce: gestisce le credenziali crittografiche, seleziona i relay, imposta i contatti e pubblica metadati (nome, biografia, immagine del profilo). Il client deve inoltre supportare flussi di pagamento e ricezione in modo intuitivo, fornendo feedback immediato sull'esito delle azioni eseguite e sulle condizioni di sicurezza associate.

2.2 Identità, chiavi e crittografia

Per autenticità e sicurezza, Nostr utilizza firme Schnorr sulla curva `secp256k1` (la stessa impiegata in Bitcoin) come meccanismo di firma digitale. All'atto della creazione del profilo, il client genera due elementi fondamentali:

- una chiave privata, destinata alla firma dei messaggi;
- una chiave pubblica, che funge da identificatore unico all'interno della rete.

In questo contesto non esistono username tradizionali: l'identità è associata alla chiave pubblica. Di conseguenza, essa è costituita dall'insieme di

chiavi e dai metadati correlati, che consentono di autenticare l'interessato e di verificare le transazioni senza ricorrere a credenziali centralizzate basate su nomi utente.

Per ricevere comunicazioni, l'utente rende nota la propria chiave pubblica, anche oltre i confini della rete Nostr. Non è necessario alcun backend centrale per gestire l'identità, poiché le credenziali rimangono interamente sotto il controllo dell'utente.

2.3 Outbox e inbox

I relay possono assumere ruoli logici distinti:

- **outbox**: pubblicazione dei messaggi dell'utente (post, commenti);
- **inbox**: ricezione di messaggi diretti o risposte.

La distinzione è logica: lo stesso relay può fungere sia da outbox sia da inbox. Un utente può scegliere di pubblicare su un relay più permissivo e ricevere da uno più selettivo, migliorando resilienza, prestazioni e controllo.

2.4 Struttura degli eventi e interoperabilità

In Nostr, ogni scambio avviene tramite un *evento*, che rappresenta un messaggio strutturato secondo un formato JSON standard. Tale formato include diversi campi fissi indispensabili per garantire validità e compatibilità tra differenti client e relay. In particolare, ogni evento possiede un identificatore unico generato da una funzione crittografica, la chiave pubblica dell'autore e, naturalmente, il contenuto stesso oltre a tanti altri campi con funzioni specifiche.¹

L'evento, una volta creato, è firmato digitalmente dall'autore per consentire a chiunque di verificarne l'autenticità. Grazie a questa struttura, i diversi client possono funzionare insieme e i messaggi possono circolare liberamente, senza dipendere da una piattaforma unica.

2.5 WebSocket, subscription e filtri

Il dialogo tra un client e un relay avviene tramite una connessione stabile chiamata *WebSocket*. Essa funziona attraverso un canale di comunicazione che, una volta aperto, rimane attivo permettendo alle prese in causa di scambiarsi messaggi in tempo reale senza dover ristabilire la connessione ogni volta. Una volta dunque stabilita la connessione, il client può chiedere

¹Per vedere la struttura di un evento Nostr si faccia riferimento all'Appendice C

al relay di inviargli solo determinati eventi, specificando un *abbonamento* (subscription) con filtri precisi, ad esempio limitati a un autore, a un certo tipo di evento o a un intervallo temporale. In questo modo non si ricevono dati superflui, ma soltanto quelli di reale interesse.

Gli scambi tra client e relay si basano su un insieme ristretto di messaggi predefiniti. Dal lato del client, i messaggi servono per inviare un evento, per aprire una subscription o per chiuderla. Dal lato del relay, invece, arrivano le risposte: il relay può trasmettere gli eventi richiesti, confermare o rifiutare quelli ricevuti, notificare la fine degli eventi disponibili o segnalare avvisi e chiusure di subscription.

In alcuni casi, i relay possono chiedere al client di autenticarsi prima di consentire l'accesso a certe risorse. Questo avviene attraverso un meccanismo di *sfida e risposta*: il relay invia una *challenge* e il client deve firmarla digitalmente per dimostrare la propria identità. Solo dopo questa verifica l'accesso viene concesso.

2.6 Estensioni del protocollo (NIP)

La struttura fondamentale di Nostr è definita dal **NIP-01**, che specifica il formato degli eventi, le regole di firma, la connessione tramite WebSocket e le operazioni di base tra client e relay. Questo NIP rappresenta il cuore del protocollo, ma da solo non è sufficiente a coprire tutte le funzionalità che gli utenti si aspettano da una piattaforma moderna.

Per ampliare le possibilità, sono stati introdotti numerosi altri NIP, ciascuno dei quali aggiunge una funzionalità specifica: messaggi privati, reazioni, supporto ai contenuti multimediali, integrazione con i pagamenti e molto altro. L'approccio scelto dalla comunità è volutamente modulare: ogni client e ogni relay può decidere quali NIP implementare, adattandosi alle proprie esigenze o alle preferenze degli utenti. Questo garantisce un'evoluzione flessibile e distribuita del protocollo, senza imporre una centralizzazione nello sviluppo.

2.6.1 Criteri di accettazione

Affinché un nuovo NIP venga riconosciuto e documentato ufficialmente, è necessario rispettare alcuni criteri di base che assicurano coerenza e utilità alla comunità. In particolare:

- deve esistere almeno una **doppia implementazione** (almeno due client e un relay) che ne dimostri la fattibilità;
- deve essere garantita la **retro-compatibilità**, in modo da non compromettere il funzionamento di client e relay che non implementano il nuovo NIP;

- la proposta deve mantenere un certo livello di **originalità**, evitando duplicazioni funzionali rispetto a NIP già esistenti.

Questi criteri riflettono la filosofia aperta e collaborativa di Nostr: ogni NIP nasce come proposta discussa pubblicamente nel repository ufficiale, e solo dopo un processo di confronto e verifica può diventare parte integrante della documentazione del protocollo.

3 Estensioni del protocollo e micropagamenti

Lo sviluppo di Nostr non si è fermato alle sole funzionalità di base. Accanto al nucleo minimo definito dal NIP-01, nel tempo la comunità ha introdotto una serie di estensioni pensate per arricchire il protocollo e renderlo adatto a diversi scenari: messaggistica privata, condivisione di contenuti multimediali, sistemi di reputazione e, soprattutto, strumenti per integrare i micropagamenti.

Questa evoluzione si collega direttamente ai limiti intrinseci di Bitcoin. La rete principale, pur essendo sicura e trasparente, non è progettata per gestire volumi elevati di transazioni né per facilitare trasferimenti rapidi e di piccolo importo. Per questo motivo è stato sviluppato il Lightning Network (LN), che introduce meccanismi *off-chain* in grado di rendere efficienti e scalabili i micropagamenti. L'integrazione di LN con Nostr ha reso possibile l'emergere di nuove funzionalità economiche, come gli *zap*, che verranno approfondite nel capitolo successivo.

In questo capitolo si fornirà dunque una panoramica sui limiti della rete Bitcoin, sulle soluzioni offerte dal Lightning Network e sui meccanismi di interazione (BOLT11, LNURL) che permettono ai micropagamenti di integrarsi in modo semplice nell'esperienza utente. Successivamente verranno illustrati i *Nostr Implementation Proposals* (NIP), lo strumento con cui la comunità definisce e standardizza le estensioni del protocollo. Questa analisi fungerà da ponte concettuale per comprendere meglio come gli *zap* rappresentino l'applicazione più significativa di tali estensioni.

3.1 Bitcoin e limiti on-chain

Il protocollo Bitcoin registra ogni transazione all'interno di una blockchain pubblica e immutabile, garantendo sicurezza, trasparenza e resistenza alla censura. Tuttavia, questo modello presenta alcuni limiti strutturali: la capacità complessiva della rete si aggira intorno a sole sette transazioni al secondo, le commissioni di rete possono crescere in maniera variabile a seconda della congestione e i tempi di conferma non sono immediati.

Questi vincoli rendono Bitcoin poco adatto ad alcuni scenari pratici, come i micropagamenti o i trasferimenti che richiedono rapidità ed efficienza. Per superare tali restrizioni, è stato sviluppato un livello aggiuntivo denominato *Lightning Network* (LN). Si tratta di un *layer 2* che opera sopra la blockchain principale e consente di aprire canali di pagamento bidirezionali *off-chain*. All'interno di questi canali, le transazioni avvengono in modo pressoché istantaneo e con costi trascurabili, mentre soltanto le operazioni di apertura e chiusura del canale devono essere registrate sulla blockchain principale.

3.2 Canali, fatture BOLT11 e schema LNURL

Il funzionamento del Lightning Network ruota attorno ai canali di pagamento *off-chain*, che permettono di effettuare transazioni rapide senza sovraccaricare la blockchain di Bitcoin. Ogni pagamento viene identificato attraverso una *fattura BOLT11*, un formato standard e firmato crittograficamente che include informazioni fondamentali come l'importo, il destinatario, l'eventuale scadenza e altri metadati. Questo meccanismo garantisce precisione e verificabilità, ma può risultare poco pratico per l'utente, poiché richiede la generazione manuale di una fattura per ogni transazione.

Per rendere l'esperienza più agevole, è stato introdotto lo schema *LNURL*, che semplifica l'interazione tra wallet e servizi Lightning. Quando parliamo di wallet (letteralmente “portafoglio”) intendiamo un'applicazione o dispositivo che consente di gestire chiavi crittografiche e di effettuare transazioni sulla rete Bitcoin o Lightning Network. Nel caso del Lightning Network, il wallet non si limita a custodire fondi, ma gestisce anche i canali di pagamento e tiene traccia del saldo associato a ciascun canale aperto con altri nodi della rete. Esistono diversi tipi di wallet Lightning:

- Wallet custodial, dove un servizio esterno conserva i fondi e gestisce i canali per conto dell'utente (più semplice da usare, ma meno sicuro).
- Wallet non custodial, dove l'utente mantiene il pieno controllo delle proprie chiavi e dei canali (più complesso ma più sicuro).

In entrambi i casi, il wallet funge da interfaccia tra l'utente e la rete Lightning, permettendo di inviare e ricevere micropagamenti quasi istantanei e a costi minimi, senza dover conoscere i dettagli tecnici del protocollo sottostante.

Tornando al punto del discorso, LNURL è un protocollo che permette di effettuare diverse operazioni nel Lightning Network — come pagamenti, richieste di fondi o autenticazioni — attraverso semplici link web (URL) leggibili. L'idea alla base è di sfruttare un meccanismo universale e già familiare, quello degli indirizzi Internet, per avviare un'interazione tra wallet e servizio remoto. Invece di scambiare complesse stringhe di dati o generare

manualmente una nuova fattura per ogni transazione, LNURL codifica le informazioni necessarie in un link (ad esempio dentro un codice QR), che il wallet può interpretare automaticamente e utilizzare per comunicare con il server.

A livello tecnico, quando un utente scansiona un codice QR LNURL (o apre un link simile), il wallet interpreta l'indirizzo contenuto nel link e contatta automaticamente il server indicato. Il server risponde inviando al wallet le informazioni necessarie per completare l'operazione — come l'importo, la descrizione del pagamento e l'indirizzo Lightning del destinatario. A questo punto, il wallet crea e invia il pagamento in modo automatico, senza che l'utente debba inserire manualmente dati o generare una nuova fattura ogni volta.

Grazie a LNURL, diventa possibile inviare fondi a un'identità leggibile, come `bob@ln.example.com`, senza dover produrre di volta in volta una nuova fattura. In questo modo, l'uso del Lightning Network si avvicina a un modello già familiare agli utenti, dove un semplice indirizzo o collegamento web diventa il punto di accesso per i pagamenti, paragonabile a un indirizzo email.

Nell'ecosistema Nostr, l'integrazione con Lightning è ulteriormente potenziata: gli utenti possono inserire il proprio indirizzo LNURL nei metadati del profilo (evento di tipo *kind 0*), nel campo `lud16`. Questo consente a chiunque di inviare uno *zap* o un pagamento semplicemente leggendo le informazioni già disponibili nel profilo, senza dover richiedere manualmente ulteriori dettagli tecnici.

Le tecnologie descritte in questo capitolo — Lightning Network, le fatture BOLT11 e lo schema LNURL — costituiscono la base tecnica necessaria per integrare i micropagamenti all'interno di Nostr. La loro applicazione più evidente e innovativa è rappresentata dal meccanismo degli *zap*, che sarà analizzato nel capitolo successivo.

4 Lightning Network (LN) e Zap

All'interno del contesto descritto nel capitolo precedente entra in gioco lo *zap*: un micropagamento effettuato tramite Lightning Network e integrato nel protocollo Nostr. In parole semplici, lo zap è un modo per inviare una piccola somma di denaro, quasi come lasciare una mancia, a chi pubblica un contenuto che si ritiene utile o interessante.

L'unione tra Nostr e Lightning Network rende questi pagamenti **istantanei, a costo minimo e senza intermediari**. Non serve cioè passare da una piattaforma centralizzata che gestisce i fondi. Questo modello non solo è rapido ed efficiente, ma rende più difficile la censura e introduce un incentivo economico: premiare direttamente chi produce contenuti di valore. È importante chiarire che lo zap non è un portafoglio digitale, ma una *funzione aggiuntiva*, descritta nel documento tecnico **NIP-57**.

4.1 Attori coinvolti

Il funzionamento degli zap non si riduce a un semplice scambio di denaro, ma coinvolge un insieme articolato di attori, ciascuno con un ruolo ben definito. Per comprendere la dinamica complessiva è utile descrivere i protagonisti principali, riassunti nella Tabella 4.1.

Ruolo	Descrizione
Alice (A)	Mittente dello zap.
Bob (B)	Destinatario dello zap; può pubblicarne la ricevuta.
Client	Applicazione (ad esempio Damus) che genera eventi zap e interagisce con Nostr.
Relay	Server che riceve e propaga eventi Nostr.
LN Node	Nodo Lightning che gestisce il canale di pagamento.
LNURL	Schema per interagire via URL con servizi o nodi Lightning; in Nostr l'indirizzo è esposto come <code>lud16</code> in kind 0.

Tabella 4.1: Attori principali nel meccanismo degli zap

Questo sistema mostra che inviare uno zap non è un gesto isolato: entrano in gioco più parti che collaborano tra loro, e proprio questa cooperazione assicura che tutto funzioni in modo sicuro e affidabile.

4.2 Sicurezza, autenticità e integrità

Per regolare il funzionamento degli zap, il NIP-57 introduce due eventi fondamentali che ne scandiscono le fasi operative:

- **kind:9734 – ZapRequest**: rappresenta l'attestazione pubblica dell'intenzione di pagamento. Contiene l'importo, un eventuale commento, la chiave pubblica del destinatario, i riferimenti all'evento che si intende zappare, l'elenco dei relay coinvolti e la firma di Alice. Pur non trasferendo denaro in sé, costituisce un elemento essenziale per garantire trasparenza e verificabilità.
- **kind:9735 – ZapReceipt**: rappresenta la conferma del pagamento. È firmato da Bob e fa riferimento alla ZapRequest originaria, fungendo da prova pubblica e verificabile che il pagamento sia effettivamente avvenuto.

Il fatto che le informazioni sugli zap siano pubbliche ha pro e contro. Da una parte è positivo, perché chiunque può controllare in modo indipendente che un pagamento sia avvenuto davvero. Dall'altra parte, però, questa stessa trasparenza può diventare un problema: vengono resi visibili dati delicati, come la cifra inviata, la frequenza dei pagamenti e i rapporti economici tra diversi utenti. In certi contesti questo può permettere di tracciare e profilare le persone. Per ridurre questo rischio, si possono usare relay privati, identità non riconducibili direttamente a una persona reale e inserire meno informazioni aggiuntive possibili, ad esempio evitando commenti.

4.3 Proprietà garantite (ipotesi forti)

Se i meccanismi crittografici e le regole del protocollo funzionano come previsto, il sistema degli zap assicura una serie di proprietà che riporto di seguito:

1. **Autenticità** – Ogni evento (9734 e 9735) è firmato con la chiave privata di chi lo crea. La firma può essere verificata con la corrispondente chiave pubblica, che garantisce l'origine del messaggio. Questa chiave pubblica si trova nei metadati (*kind 0*) oppure può essere validata tramite NIP-05, che collega una chiave a un dominio DNS. Non è semplice ricondurla subito a una persona reale, ma strumenti come DNSSEC, firme PGP collegate o la reputazione aiutano a renderla più affidabile.

2. **Integrità** – Ogni evento ha un identificatore *id*, calcolato tramite hash sui suoi campi principali. Insieme alla firma digitale, questo impedisce di modificare il contenuto senza che la manomissione venga rilevata.
3. **Non ripudio** – Chi invia una *ZapRequest* non può poi negarlo, perché porta la sua firma; allo stesso modo, chi riceve uno zap non può negare una *ZapReceipt* che è già stata diffusa e salvata nei relay.
4. **Verificabilità pubblica** – Grazie ai riferimenti diretti all'id della richiesta e alla chiave pubblica di chi l'ha inviata, chiunque può controllare che una determinata *ZapReceipt* corrisponda davvero alla sua *ZapRequest*, interrogando i relay.
5. **Resistenza alla censura** – Essendo distribuito, Nostr non si blocca se un relay decide di non diffondere un evento: altri relay continueranno a farlo, rendendo quasi impossibile una censura completa.

4.4 Analisi in condizioni forti

Alla luce delle proprietà appena descritte, è possibile affermare che, nelle condizioni ipotizzate, il meccanismo degli zap realizza effettivamente gli obiettivi di sicurezza dichiarati. La pubblicazione della ricevuta, in particolare, non ha soltanto un valore informativo, ma costituisce uno strumento fondamentale: consente a client, relay e terze parti di verificare l'avvenuto pagamento, di sbloccare contenuti eventualmente condizionati e di alimentare metriche di reputazione all'interno dell'ecosistema Nostr.

La trasparenza, tuttavia, comporta un costo in termini di riservatezza. L'analisi sistematica degli eventi pubblici può rivelare legami economici tra identità, oltre a frequenza e importi dei trasferimenti, e persino il contesto (post o evento) a cui tali pagamenti si riferiscono. In scenari delicati, questo può tradursi in forme sofisticate di profilazione, sollevando interrogativi sulla tutela della privacy.

Dimostrazione sintetica. Alla luce delle ipotesi forti, attacchi come lo spoofing degli eventi, la generazione di invoice fraudolente o la compromissione di bot delegati risultano poco praticabili. Ciò è dovuto a tre fattori principali:

- ogni evento è firmato crittograficamente;
- ogni invoice è valida soltanto se emessa da un nodo Lightning legittimo;
- i bot operano esclusivamente entro i limiti definiti da una delega firmata (NIP-26).

Limiti residui. Nonostante i punti di forza, permangono alcune vulnerabilità strutturali che meritano attenzione:

- una ZapReceipt può non essere pubblicata dal destinatario;
- la correlazione tra richiesta e ricevuta può andare persa qualora i relay siano disgiunti;
- i metadati transazionali rimangono comunque esposti pubblicamente;
- la pubblicazione della ricevuta dipende in ultima istanza dalla cooperazione del destinatario, il che significa che, in mancanza di tale passo, la prova del pagamento resta confinata al solo livello Lightning Network.

Questi limiti mostrano che, nonostante l'integrazione tra Nostr e Lightning Network rappresenti un passo importante verso micropagamenti decentralizzati e trasparenti, persistono alcune criticità legate a trasparenza, verificabilità e privacy. Una discussione più approfondita di tali vulnerabilità, insieme alle possibili contromisure tecniche, è presentata nel Capitolo 6, Analisi delle Vulnerabilità. In sintesi, gli zap trasformano Nostr da una semplice rete sociale distribuita a un ecosistema economico dove gli utenti possono sostenere i contenuti con micro-pagamenti veloci, sicuri e pubblicamente verificabili.

I vantaggi sono chiari: autenticità, sicurezza e resistenza alla censura. Ma allo stesso tempo, la trasparenza introduce sfide per la privacy. Questo equilibrio tra apertura e riservatezza è uno dei temi di ricerca più attuali e in evoluzione.

Il capitolo successivo descriverà due simulazioni pratiche — una in ambiente non-custodial e una in ambiente custodial — per confrontare i diversi scenari di gestione dei pagamenti.

5 Simulazione tecnica

Dopo aver delineato il funzionamento teorico degli zap, ho ritenuto opportuno realizzare una sperimentazione pratica per verificarne le proprietà in un ambiente controllato. L'obiettivo non è soltanto confermare la correttezza dei meccanismi di sicurezza e affidabilità descritti nei capitoli precedenti, ma anche evidenziare risultati e effetti concreti che emergono nella gestione dei micropagamenti: semplicità d'uso, livello di autonomia dell'utente e grado di dipendenza da terze parti.

A tal fine sono stati condotti due esperimenti complementari, che incarnano modelli opposti di gestione delle risorse: lo **scenario non-custodial**, nel quale ciascun attore mantiene il pieno controllo delle proprie chiavi e dei propri fondi, e lo **scenario custodial**, dove la responsabilità dell'infrastruttura viene delegata a un provider esterno.

5.1 Scenario non-custodial: Polar

5.1.1 Ambiente

Per simulare l'approccio non-custodial si è scelto di utilizzare **Polar**, un framework open-source che consente di creare rapidamente un ambiente di test per Lightning Network. Con Polar è possibile avviare in pochi minuti più nodi Bitcoin e Lightning, configurare wallet e stabilire connessioni, ottenendo così una rete completa ma confinata in locale.

La topologia minima adottata comprende:

- due nodi LND (Alice e Bob), ciascuno associato al proprio nodo Bitcoin;
- un nodo `bitcoind` in modalità *regtest*, con un miner locale capace di generare blocchi istantaneamente per confermare le transazioni;
- la possibilità di aggiungere ulteriori nodi per simulare altri attori.

Questa configurazione permette di riprodurre con precisione il funzionamento tecnico del Lightning Network, pur senza includere nativamente componenti del protocollo Nostr. Una simulazione completa degli zap richiederebbe infatti l'aggiunta di script esterni in grado di pubblicare gli eventi

9734 e 9735 su un relay locale. In questa fase si è scelto di concentrarsi principalmente sul cuore dei pagamenti Lightning.

5.1.2 Setup, funding e apertura del canale

L'esperimento si propone di eseguire uno zap nella sua forma più basilare: un pagamento off-chain tra Alice e Bob, con discovery tramite LNURL-pay e fattura BOLT11.

I componenti utilizzati sono:

- Polar come ambiente di orchestrazione (basato su Docker);
- due nodi LND (Alice e Bob);
- un nodo `bitcoind` in regtest, che genera la blockchain privata;
- una connessione P2P diretta Alice–Bob, necessaria per l'apertura del canale LN.

Fase 1 – Reperimento delle informazioni. Il client di Alice ottiene l'indirizzo Lightning (`lud16`) di Bob dai metadati pubblicati nel suo profilo Nostr (evento *kind:0*):

```
{
  "name": "Bob",
  "about": "Developer",
  "picture": "https://...",
  "lud16": "bob@ln.example.com"
}
```

Questa informazione è il punto di partenza per il flusso LNURL-pay: il wallet interroga l'endpoint remoto associato a Bob e riceve in risposta una fattura BOLT11. In questo contesto, Nostr funge soltanto da strato di *discovery*, mentre l'esecuzione del pagamento resta interamente confinata a LN.

Fase 2 – Creazione della ZapRequest. Alice genera e firma un evento `kind:9734`, dichiarando la propria intenzione di pagare:

```
{
  "kind": 9734,
  "pubkey": "pubkey_di_alice",
  "content": "Grazie per il tuo lavoro!",
  "tags": [
    ["p", "pubkey_di_bob"],
    ["e", "id_evento_originale"],
    ["relays", "wss://relay.nostr.example"],
    ["amount", "1000"]
  ],
  "created_at": 1680000000,
  "sig": "firma_digitale"
}
```

L'importo, espresso in *millisatoshi*, compare nel tag `amount`. La firma digitale copre l'intero oggetto JSON (eccetto `id` e `sig`), garantendo l'immutabilità dei dati. Un eventuale attacco di replay è mitigato dai relay, che eliminano duplicati tramite l'`id`.

Fase 3 – Pagamento Lightning. Il flusso LNURL-pay segue tre passaggi principali:

1. richiesta a `https://ln.example.com/.well-known/lnurlp/bob`, che restituisce parametri come `minSendable`, `maxSendable`, `metadata`, `callback`;
2. invio al `callback` dell'importo desiderato;
3. ricezione della fattura BOLT11, successivamente pagata da Alice.

Fase 4 – Pubblicazione della ricevuta. Dopo aver ricevuto il pagamento, Bob (o un suo bot) verifica la fattura e pubblica un evento `kind:9735` come prova pubblica:

```
{
  "kind": 9735,
  "pubkey": "pubkey_di_bob",
  "tags": [
    ["e", "id_evento_zaprequest"],
    ["p", "pubkey_di_alice"]
  ],
  "created_at": 1680001234,
  "sig": "firma_digitale_bob"
}
```

Fase 5 – Simulazione pratica in Polar. Il test si completa con l'esecuzione di comandi sui nodi LND e Bitcoin:

1. creazione di un indirizzo on-chain e funding da parte di `bitcoind`;
2. mining immediato per confermare il funding;
3. connessione P2P tra Alice e Bob;
4. apertura del canale LN con `lncli openchannel`;
5. generazione della fattura BOLT11 da parte di Bob e pagamento da parte di Alice;
6. verifica del bilancio canale con `lncli channelbalance`.

L'output finale mostra che la transazione è stata eseguita con successo, con fee prossime allo zero e restituzione del *preimage*, la prova crittografica della corretta esecuzione del pagamento.

5.2 Scenario custodial: LNbits e Zeus Embedded LND

Per rappresentare il modello opposto è stato scelto un approccio custodial. Il nodo ricevente è ospitato su **LNbits** (demo testnet), mentre il mittente utilizza **Zeus Wallet** in modalità Embedded LND. In questo scenario, chiavi e canali sono gestiti interamente dal provider.

La procedura è estremamente semplificata:

1. creazione del wallet su LNbits, con generazione delle chiavi Admin/Invoice;
2. installazione e configurazione di Zeus Wallet in testnet;
3. ottenimento di fondi di prova tramite faucet e conferma on-chain;
4. scansione del QR code di LNbits e pagamento immediato della fattura.

Il pagamento si conclude in tempi pressoché istantanei, grazie alla liquidità già disponibile sui canali del provider. L'esperienza utente risulta molto più lineare rispetto al caso non-custodial, sebbene al prezzo di un minor controllo sui fondi.

5.3 Confronto tra modelli

I due esperimenti mettono chiaramente in luce i rispettivi vantaggi e limiti.

Modello	Controllo chiavi	Setup/complessità	Velocità esecuzione
Custodial	Chiavi detenute dal provider	Molto semplice: creazione wallet e generazione fattura quasi istantanee	Molto alta: il provider ha già canali aperti e liquidità disponibile
Non-custodial	Chiavi sotto esclusivo controllo dell'utente	Più complesso: richiede avvio dei nodi, funding on-chain, apertura canali e configurazione client/relay	Più lenta all'avvio, perché dipende dalla conferma on-chain del funding; una volta aperti i canali, i pagamenti off-chain sono rapidi

Tabella 5.2: Confronto: chiavi, complessità e velocità

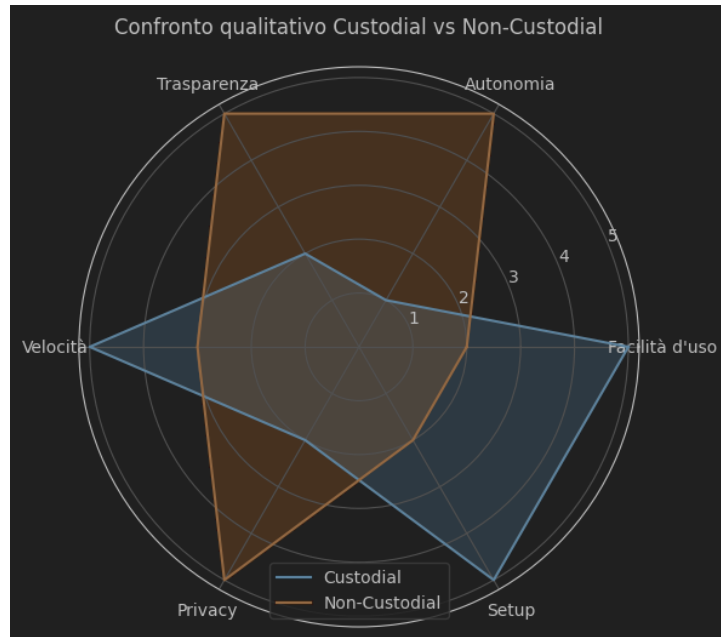


Figura 5.1: Confronto qualitativo Custodial/Non-Custodial

Modello	Dipendenza da terzi	Trasparenza/visibilità
Custodial	Elevata: il provider è punto di fiducia e possibile vulnerabilità	Limitata a quanto esposto dal servizio
Non-custodial	Assente: nessun intermediario	Massima: pieno accesso a log e dettagli

Tabella 5.4: Confronto: dipendenza e trasparenza

Il modello non-custodial offre autonomia e trasparenza, ma richiede competenze tecniche e maggiore complessità iniziale. Lo scenario custodial garantisce invece una fruizione immediata, a scapito però di controllo e indipendenza. La scelta tra i due dipende dal contesto applicativo, dal livello di fiducia verso terze parti e dalla dimestichezza tecnica degli utenti.

5.4 Mini-valutazione quantitativa

Per completare la sperimentazione è stata introdotta anche una valutazione quantitativa, finalizzata a misurare le prestazioni percepite dagli utenti durante l'esecuzione di uno zap.

Protocollo di misura ($n \geq 10$)

1. Strumentare il client per loggare due timestamp: `t0_click` (pressione del bottone Zap) e `t1_receipt` (ricezione del primo evento 9735 reso nel DOM).
2. Per ogni run: generare l'evento 9734, eseguire il flusso LNURL $\rightarrow$ BOLT11, effettuare il pagamento e attendere l'evento 9735.
3. Calcolare la latenza percepita come **TTFZ** = `t1_receipt` - `t0_click`.
4. Registrare, oltre al TTFZ, anche: *esito* (ok/ko), *fee* (in satoshi), percorso dei relay (A,B,...), wallet utilizzato e importo.
5. Su un campione di n misurazioni valide, calcolare media, deviazione standard e varianza della latenza.

Risultati preliminari

I valori riportati di seguito derivano da misurazioni indicative in ambiente di test e da dati riportati in letteratura e sono quindi da considerarsi stime attendibili. Sono stati analizzati due scenari contrapposti.

Scenario non-custodial (Polar/LND). Il setup richiede **5 passi**:

1. funding su regtest;
2. connessione tra nodi;
3. apertura di un canale;
4. deploy del client/backend;
5. configurazione del relay.

Il tempo medio per arrivare a zero (*TTFZ*) si attesta attorno a **3–5 secondi**, con una varianza di circa **1,2 secondi**. Il tasso di successo delle transazioni nei test è stimato intorno al **98–99%**, in linea con quanto riportato da studi recenti sul Lightning Network [1]. Le fee rilevate sono prossime allo zero, tipicamente nell'ordine di **0–1 satoshi**.

Scenario custodial (LNbits + wallet). Il setup richiede **3 passi**:

1. creazione del wallet LNbits;
2. configurazione del wallet client (Zeus o Alby);
3. configurazione del relay/client.

In questo contesto il tempo medio $TTFZ$ è leggermente inferiore, attorno a **2–3 secondi**, grazie alla disponibilità immediata di liquidità sui canali del provider. La varianza della latenza risulta contenuta (**0,8 secondi** circa), mentre il tasso di successo si conferma elevato, nell'ordine del **99%**. Le fee risultano leggermente più alte rispetto al modello non-custodial, tra **0–3 satoshi**, ma comunque trascurabili per importi tipici dei micropagamenti.

Metrica	Valore medio	Fonte
Tempo medio pagamento LN (1–3 hop)	0,4–2,3 s	[1]
Tempo medio pagamento LN (5+ hop)	4–6 s	[1]
Tasso di successo micropagamenti LN	98–99%	river2022 , [1]
Fee micropagamenti LN	0–3 sat (0,02–0,04%)	acinq2023 , [1]
Latenza media Zap Receipt (Nostr)	2–6 s	[2]
Throughput relay Nostr (SQLite)	~100 req/s	[3]

Tabella 5.5: Sintesi delle prestazioni di Lightning Network e Nostr riportate in letteratura.

Questi risultati preliminari confermano le differenze attese: il modello non-custodial offre pieno controllo delle chiavi a scapito di maggiore complessità iniziale, mentre quello custodial privilegia semplicità d'uso e tempi leggermente più rapidi, al prezzo di una fiducia maggiore verso il provider.

Conclusioni

La sperimentazione dimostra come i due modelli incarnino filosofie opposte: il non-custodial valorizza autonomia e controllo diretto, mentre il custodial privilegia la semplicità. Entrambi consentono di realizzare zap funzionanti e sicuri, ma con implicazioni differenti in termini di trasparenza, fiducia e usabilità.

Questi esperimenti costituiscono dunque una validazione pratica delle proprietà teoriche discusse nei capitoli precedenti e forniscono un banco di prova per la valutazione dell'adozione degli zap in scenari reali, dove le esigenze degli utenti possono variare notevolmente.

6 Analisi delle vulnerabilità

Quando si parla di zap (i micropagamenti all'interno del protocollo Nostr) e di pagamenti tramite Lightning Network, è importante capire che ci sono diversi possibili punti deboli. Alcuni problemi possono verificarsi anche se tutto funziona in modo corretto e nessuno ha rubato chiavi o preso il controllo di dispositivi. Altri, invece, emergono solo se vengono toccati elementi considerati “sicuri”, come chiavi private, client, relay o le comunicazioni.

In questo capitolo illustriamo i principali scenari di attacco, spiegando quali contromisure tecniche si possono adottare.

6.1 Attacchi possibili anche con ipotesi forti

Ci sono situazioni in cui, anche senza la compromissione di chiavi o dispositivi, il sistema può perdere trasparenza e diventare difficile da verificare. Un primo esempio riguarda i cosiddetti *relay disgiunti*. Può accadere che la richiesta di zap (**ZapRequest**) venga pubblicata su un relay e la corrispondente ricevuta (**ZapReceipt**) su un altro. In questo caso, chi consulta un solo relay non riesce a osservare l'intera interazione. Interrogare più relay contemporaneamente è pratica comune in Nostr, ma se richiesta e ricevuta non compaiono mai insieme almeno su uno di essi, l'associazione tra le due rimane invisibile, anche per chi interroga più fonti.

Un secondo rischio è legato alla mancata pubblicazione della ricevuta. Immaginiamo che Bob riceva i fondi su Lightning Network, ma decida di non pubblicare su Nostr l'evento che conferma il pagamento (**kind:9735**). In questo scenario, la prova del pagamento resta registrata nel sistema Lightning, ma non è più collegabile alla **ZapRequest** che lo ha originato. Alice, che ha inviato i fondi, non può supplire a questa mancanza: anche se possiede la prova crittografica del pagamento (il *preimage* di Lightning), non esiste alcun legame diretto con l'evento di richiesta pubblicato su Nostr. Solo Bob, firmando con la propria chiave Nostr, può creare una **ZapReceipt** valida e verificabile, e senza il suo intervento il collegamento resta spezzato.

Infine, occorre considerare il problema dello *spam* di **ZapRequest**. Un attore malevolo può infatti inviare in massa eventi **kind:9734** senza completare i relativi pagamenti. Questo comportamento produce rumore, può saturare i relay e compromette l'esperienza degli utenti legittimi. Si tratta di

un classico attacco di flooding, che richiede opportune misure di mitigazione per proteggere l'infrastruttura e mantenere il sistema usabile.

6.2 Attacchi che richiedono il venir meno delle ipotesi forti

Esiste una seconda categoria di attacchi che si manifesta quando alcune condizioni di sicurezza di base vengono meno: ad esempio se una chiave privata viene rubata, se un client è vulnerabile, se un relay è controllato da un attaccante oppure se le comunicazioni non sono protette. In questi casi gli scenari diventano ancora più critici.

Un primo rischio è lo *spoofing* del campo `lud16`. Un profilo Nostr può infatti dichiarare un indirizzo LNURL falso, che indirizza i pagamenti verso un server controllato dall'attaccante. Le tecniche usate sono spesso semplici ma efficaci: indirizzi molto simili a quelli reali (*typo-squatting*), account compromessi (*hijacking*), oppure bot automatici che reindirizzano i micropagamenti verso nodi malevoli.

Un altro scenario riguarda l'intercettazione delle comunicazioni LNURL da parte di un attacco *man-in-the-middle* (MITM). Un aggressore può manipolare la risposta LNURL e cambiare i metadati o la fattura, facendola risultare comunque valida ma firmata da un nodo sotto il suo controllo. In questo caso, non esiste più alcun legame crittografico con la chiave pubblica Nostr del vero destinatario.

Un rischio ulteriore è quello della compromissione del client. Se la chiave privata dell'utente viene conservata in modo insicuro — ad esempio in un'app web che la salva in `localStorage`, o in un'app mobile priva di protezioni locali come PIN o biometria — un attaccante può sottrarla ed emettere eventi fraudolenti a nome della vittima.

Da non sottovalutare è anche il problema del *replay* delle ZapRequest. Un evento `kind:9734`, seppur firmato correttamente, può essere semplicemente ripubblicato su altri relay. La firma resta valida perché l'evento non viene modificato, ma duplicato. Non esistendo un identificativo univoco obbligatorio, questa pratica genera rumore e confusione senza corrispondere a una nuova richiesta autentica, segnalando così la necessità di introdurre marker univoci.

Un ulteriore rischio è lo *spoofing* della fattura (invoice). Un server LNURL malevolo o compromesso può infatti emettere fatture che in realtà appartengono a nodi terzi, con il risultato di dirottare i pagamenti verso destinatari non autorizzati.

Sempre legato alla comunicazione vi è il pericolo di un MITM sul pagamento. Se la connessione avviene tramite protocollo insicuro (`http://`), un aggressore che si trova “in mezzo” — come un captive portal o un proxy manipolato — può riscrivere campi sensibili come `callback`, `invoice` o

`metadata`, restituendo una fattura generata dal proprio nodo. Se invece la connessione usa `https://` con certificati validi, questo tipo di attacco non è praticabile, perché la manipolazione verrebbe rilevata dal client.

Infine, c'è il rischio del bot delegato compromesso. Se a un bot vengono concessi permessi troppo ampi, questo può abusare della delega: ad esempio pubblicando ricevute false o impersonando l'identità del delegante. È quindi fondamentale limitare con precisione i poteri della delega e definirne chiaramente gli ambiti di validità.

6.3 Contromisure tecniche

Per ridurre i rischi descritti in precedenza, sono state individuate diverse contromisure tecniche, distribuite lungo le principali fasi operative del processo di pagamento e di pubblicazione degli eventi su Nostr.

6.3.1 Reperimento dell'indirizzo LNURL

Un primo aspetto riguarda il modo in cui viene recuperato l'indirizzo LNURL. È importante rafforzare il legame tra il dominio dichiarato e l'identità pubblica Nostr dell'utente. Ciò può avvenire tramite DNSSEC e record TXT che contengano la chiave pubblica, oppure con una *firma incrociata con PGP* inserita nei metadati del profilo (`kind:0`). In questo modo i client possono verificare l'autenticità del campo `lud16` e mostrare un badge "Verificato via PGP". A questi strumenti possono affiancarsi meccanismi di reputazione locali, ad esempio valutando lo storico degli zap, la percentuale di ricevute pubblicate o eventuali segnalazioni negative. È inoltre essenziale rifiutare indirizzi LNURL serviti su `http://` e controllare la coerenza tra i metadati restituiti e le informazioni presenti nel profilo Nostr.

6.3.2 Emissione della ZapRequest

Al momento dell'invio di una ZapRequest, il client dovrebbe sempre chiedere una conferma esplicita all'utente, mostrando chiaramente il destinatario, l'importo, i relay coinvolti e il messaggio. Per prevenire attacchi di *replay*, il campo `created_at` dovrebbe avere una validità temporale limitata (ad esempio ± 5 minuti). Inoltre, l'uso di un tag univoco come `["zap_id", <uuid>]` consente sia ai relay che ai client di eliminare i duplicati. Dal lato relay, conviene eliminare richieste con lo stesso `id`, mentre dal lato client è utile aggregare le notifiche per evitare confusione.

6.3.3 Pagamento via LNURL

Durante la fase di pagamento, la sicurezza si basa soprattutto sulla verifica del dominio. Il campo `callback` deve corrispondere al dominio dichiarato

nel campo `lud16`. È obbligatorio l'uso di connessioni `https://` con certificati TLS validi, così da impedire modifiche non autorizzate ai dati. Una misura aggiuntiva può essere la firma crittografica dei metadati restituiti dall'LNURL, confrontandoli poi con quelli presenti nel profilo Nostr. Anche in questo caso, DNSSEC può fungere da ulteriore garanzia di collegamento tra dominio e identità pubblica.

6.3.4 Emissione della ZapReceipt

Per quanto riguarda la ricevuta, il client dovrebbe distinguere chiaramente tra stato *pending* e *confirmed*. Se entro 5–10 minuti non viene pubblicata la ricevuta associata, lo zap resta in *pending*, e viene confermato solo alla comparsa della **ZapReceipt**. I relay possono introdurre politiche che incentivino la pubblicazione delle ricevute, ad esempio promuovendo le richieste che hanno una ricevuta associata. I client, a loro volta, possono calcolare punteggi reputazionali basati sulla percentuale di ricevute pubblicate e sui tempi medi di pubblicazione. Inoltre, il protocollo NIP-26 consente di gestire deleghe sicure: il delegante firma delle condizioni che il delegato deve includere nei propri eventi, senza mai cedere la chiave privata principale. In questo modo il delegato può emettere soltanto eventi specifici (come `kind:9735`), entro limiti temporali e quantitativi ben definiti.

6.3.5 Visibilità tra relay

Un'ulteriore misura riguarda la visibilità tra i relay. È utile includere nei tag della **ZapRequest** gli indirizzi dei relay previsti, così da facilitare l'individuazione delle ricevute. Relay *bridge* o *mirror* possono replicare gli eventi su più nodi, aumentando la probabilità che richiesta e ricevuta compaiano insieme. I client, inoltre, possono avvisare l'utente quando una ricevuta è stata pubblicata su un relay non previsto, suggerendo di consultarlo. Per contrastare lo *flooding*, i relay possono applicare meccanismi di *rate limiting*, ad esempio limitando a cinque ZapRequest al minuto per ogni chiave pubblica. Infine, i client possono filtrare gli zap, mostrando per impostazione predefinita soltanto quelli che dispongono di una ricevuta, mentre gli altri rimangono contrassegnati come “in attesa”.

6.4 Rischi chiave e mitigazioni

La Tabella 6.2 riassume i rischi principali, gli effetti attesi e le mitigazioni pratiche suggerite.

Rischio	Effetto	Mitigazione pratica
Relay disgiunti (9734 su A, 9735 su B)	Perdita di correlazione e verificabilità	Inclusione del tag ["relays", ...], utilizzo di bridge/mirror selettivi, ricerca mirata delle ricevute
Rifiuto della 9735	Pagamento non attestato su Nostr	Gestione dello stato <i>pending/confirmed</i> , policy relay favorevoli, punteggi reputazionali
Spoof <code>lud16</code> / MITM LNURL	Fatture o metadata alterati	HTTPS obbligatorio, verifica dominio <code>callback</code> ↔ <code>lud16</code> , uso di DNSSEC e warning su mismatch
Replay o flood di 9734	Rumore e spam	Validità temporale di <code>created_at</code> , tag univoco <code>zap_id</code> , deduplicazione lato relay e lato client
Bot delegato compromesso	Ricevute false o abusi	Uso di NIP-26 con scope ristretto, scadenza, rate-limit e audit delle deleghe

Tabella 6.2: Sintesi dei rischi chiave e delle mitigazioni proposte

6.5 Considerazioni di sicurezza

Le minacce più significative non derivano dalla crittografia o dall'architettura del protocollo, ma piuttosto da fattori esterni: uso scorretto dei client, affidamento a metadati non verificati, frammentazione tra relay e comportamenti malevoli o poco cooperativi. Con buone pratiche di progettazione, implementazioni attente e strumenti di reputazione e validazione incrociata, l'esperienza può rimanere sicura e affidabile. In queste condizioni, gli zap riescono a realizzare in modo robusto il modello di micropagamento decentralizzato previsto da Nostr.

Parte II

Applicazioni e Modelli di
Business

7 Applicazioni, Modelli di Business e ZapGate

Dopo aver esaminato nella prima parte della tesi i principi tecnici, le questioni di sicurezza e i risultati preliminari, questa seconda parte si concentra sugli aspetti applicativi e sui modelli di business che gli zap possono rendere possibili all'interno dell'ecosistema Nostr. A supporto di questa analisi viene inoltre presentato ZapGate, un prototipo sviluppato per dimostrare in modo concreto come integrare i micropagamenti nel protocollo.

L'introduzione degli zap segna un momento di svolta: non si tratta soltanto di un miglioramento tecnologico basato sull'uso di micropagamenti via Lightning Network, ma di un vero e proprio strumento per la nascita di nuove economie digitali. Queste economie possono funzionare senza intermediari centralizzati, con costi di transazione quasi nulli, e soprattutto possono abilitare modelli di interazione economica che fino a poco tempo fa erano poco praticabili o non convenienti da implementare.

7.1 Zap come abilitante per nuovi modelli economici

Nei modelli economici tradizionali, anche un pagamento di pochi centesimi comporta costi fissi sproporzionati, dovuti a commissioni minime, costi di gateway o oneri di riconciliazione. Gli *Zap*, invece, consentono il trasferimento di valore in tempo reale per importi irrisori, fino a frazioni di centesimo, senza costi aggiuntivi significativi.

Questo rende sostenibili una serie di scenari che prima erano difficilmente attuabili, come ad esempio:

- **ricompense istantanee**, attribuite per singoli contributi di valore (un post, un commento, una correzione);
- modelli **pay-per-view** per contenuti multimediali o articoli, che evitano di imporre abbonamenti ricorrenti;
- **donazioni micro-frazionate**, distribuite tra più creatori nel corso di un'unica sessione di utilizzo;

- **incentivazione di azioni** specifiche (ad esempio compilazione di un sondaggio o segnalazione di bug), premiando l'utente con micropagamenti immediati.

Questi esempi mostrano come anche contributi marginali possano acquisire un valore economico riconoscibile e trasferibile, favorendo un ecosistema più partecipativo e sostenibile.

7.1.1 Integrazione nei client

L'efficacia degli *Zap* dipende fortemente dalla loro integrazione nei client Nostr. Un'implementazione ben progettata deve rendere l'esperienza fluida e coerente, riducendo al minimo la complessità percepita dall'utente. In particolare, un client dovrebbe:

- **rilevare automaticamente** la presenza di un indirizzo LNURL/lud16 nei metadati del profilo (`kind:0`), evitando inserimenti manuali;
- offrire un'**interfaccia di pagamento** semplice e uniforme, con importi suggeriti ma anche la possibilità di personalizzare la cifra;
- **gestire in background** la firma e la pubblicazione dell'evento `kind:9734`, nascondendo i dettagli tecnici agli utenti meno esperti;
- garantire l'**interoperabilità con diversi wallet**, rispettando la scelta dell'utente tra soluzioni custodial e non-custodial;
- interpretare la **ZapReceipt** (`kind:9735`) come conferma crittografica, sbloccando automaticamente eventuali contenuti o funzionalità collegate.

Nei client più maturi questo flusso diventa quasi invisibile: all'utente basta premere un pulsante e la transazione si completa in meno di un secondo, mantenendo al contempo tracciabilità e verificabilità.

7.1.2 Rimozione di intermediari

Uno dei vantaggi più rilevanti degli *Zap* è la possibilità di eliminare del tutto i tradizionali gateway di pagamento centralizzati, con le loro commissioni e restrizioni. Non vi è più la necessità di ricorrere a sistemi di accredito e prelievo legati a banche o carte di credito, né di sottostare a procedure di registrazione su piattaforme di terze parti.

Questo consente a una platea molto ampia di attori — dal singolo creatore di contenuti, alla redazione giornalistica, fino ai piccoli marketplace o alle community online — di ricevere pagamenti direttamente, senza barriere

geografiche o istituzionali. La riduzione della complessità tecnica e burocratica apre la strada a micro-economie native del protocollo, che vivono e prosperano interamente all'interno dell'ecosistema Nostr, senza la necessità di appoggiarsi a infrastrutture tradizionali, se non per la conversione eventuale in valuta fiat.

Tali servizi possono essere offerti sia in modalità *pay-per-use* sia in forma di abbonamento, ed essere integrati direttamente nei client o forniti tramite plugin esterni. In entrambi i casi, la logica rimane quella di ridurre al minimo gli intermediari e massimizzare il valore diretto trasferito tra gli attori.

8 ZapGate: applicazione concreta

Dopo aver discusso i modelli economici abilitati dagli *Zap*, questo capitolo presenta il prototipo **ZapGate**, concepito come *proof-of-concept* capace di dimostrare in maniera concreta come l'integrazione tra Nostr e Lightning Network possa dare vita a un ecosistema sociale e finanziario realmente distribuito. L'idea alla base del progetto è quella di creare una piattaforma che non si limiti a ipotizzare scenari, ma permetta agli utenti di sperimentare direttamente la pubblicazione e la monetizzazione di contenuti digitali attraverso micropagamenti verificabili.

8.1 Cos'è una “killer app”

Zapgate può essere definita una vera e propria killer app, ossia un'applicazione utile, semplice e immediata che, essendo provata, possa favorire in maniera decisiva l'adozione di questa tecnologia. Nel contesto di Nostr e degli Zap, una killer app dovrebbe essere in grado di mostrare in modo chiaro e pratico i benefici dei micropagamenti decentralizzati, rendendoli accessibili non solo dal punto di vista tecnico ma anche come esperienza d'uso. In questo senso, nello sviluppo di ZapGate ho proprio cercato di dare questa direzione: altro non è che un prototipo funzionante e che consente di testare tutte le funzionalità necessarie per sfruttare al massimo la tecnologia Zap.

8.2 Perché ZapGate? Obiettivi e funzionalità

ZapGate nasce con l'intento di proporre un modello di monetizzazione diretto, trasparente e decentralizzato, basato sull'ecosistema Nostr e sull'integrazione con la rete Lightning. L'obiettivo principale è quello di consentire ai creatori di contenuti di ricevere compensi in modo immediato, senza la necessità di intermediari, e di garantire agli utenti un'esperienza di pagamento semplice, sicura e a basso costo. In questa prospettiva, ogni contenuto pubblicato su Nostr può essere reso accessibile soltanto a seguito dell'invio di uno Zap verificato, che rappresenta la prova di un micropagamento effettuato tramite Lightning Network.

Il modello proposto da ZapGate si fonda su un meccanismo di pubblicazione **a due livelli**. Ogni contenuto è inizialmente disponibile in forma di anteprima pubblica, pensata per fornire una panoramica o un estratto del materiale completo. L'accesso integrale, invece, è subordinato all'effettuazione di un piccolo pagamento, che consente lo sblocco immediato del contenuto una volta completata la transazione. Questo approccio mira a coniugare la logica della condivisione aperta, tipica del protocollo Nostr, con la possibilità per i creatori di ottenere una remunerazione proporzionata al valore percepito dai lettori.

Il sistema di sblocco automatico costituisce uno degli elementi più innovativi del prototipo: al termine della transazione Lightning, il contenuto viene reso immediatamente disponibile all'utente, senza ulteriori passaggi o conferme manuali. Tale automatismo è reso possibile dall'integrazione tra il client Nostr e l'infrastruttura Lightning, che consente di verificare in tempo reale l'avvenuto pagamento.

Un altro aspetto rilevante riguarda la compatibilità con diversi tipi di wallet Lightning. ZapGate è stato concepito per supportare sia soluzioni non-custodial, come Phoenix, Zeus o Alby, sia wallet custodial integrati, lasciando all'utente la piena libertà di scelta in base alle proprie preferenze in termini di sicurezza e praticità. In questo modo, il sistema non impone un'infrastruttura proprietaria, ma si adatta all'ecosistema già esistente.

Per quanto riguarda le modalità di pagamento, ZapGate offre due approcci complementari. Nel caso dell'utilizzo di un wallet esterno non-custodial, l'utente effettua il pagamento aprendo un'app Lightning dedicata e confermando manualmente la transazione. Questa modalità garantisce il pieno controllo dei fondi da parte dell'utente, in linea con i principi di decentralizzazione e sovranità individuale. In alternativa, quando si utilizza un wallet integrato di tipo custodial, come nel caso dell'estensione Alby, il pagamento può avvenire in modo completamente trasparente attraverso le API del wallet, previa autorizzazione esplicita dell'utente.

In entrambe le configurazioni, il client Nostr non gestisce mai direttamente i fondi: il ruolo di intermediario tecnico è sempre affidato al wallet Lightning selezionato, che esegue le operazioni necessarie senza introdurre elementi di centralizzazione. Ciò consente di mantenere la coerenza con la filosofia aperta e distribuita del protocollo Nostr, offrendo al contempo un'esperienza d'uso fluida e affidabile.

8.3 Architettura generale

L'architettura di *ZapGate* unisce componenti decentralizzati e un backend molto leggero, con l'obiettivo di mantenere il sistema semplice ma allo stesso tempo stabile e sicuro. Tutto il progetto è stato pensato per rispetta-

re i principi del protocollo Nostr, evitando elementi di centralizzazione e privilegiando la trasparenza delle comunicazioni.

Dal punto di vista strutturale, l'applicazione è composta da cinque parti principali: il **frontend**, i **relay Nostr**, i **nodi Lightning**, le **API backend** e lo **storage locale**. Ognuna di queste componenti ha un ruolo specifico nel processo di pubblicazione, trasmissione e monetizzazione dei contenuti.

Il **frontend** è stato realizzato con *Next.js* e *React*, utilizzando *TailwindCSS* per la gestione dello stile. Questa parte gestisce l'interfaccia grafica e le principali interazioni dell'utente. Attraverso il client è possibile creare nuovi post, visualizzare il feed, inviare e ricevere *Zap*, oltre ad accedere alle sezioni dedicate ai trend, ai creator seguiti e alla gestione del proprio profilo.

La comunicazione tra gli utenti avviene tramite una rete di **relay Nostr**, configurati localmente. Nel prototipo sono stati usati tre relay principali (*Relay1*, *Relay2* e *Relay3*), che si occupano di distribuire e sincronizzare gli eventi generati dal sistema. Tra questi rientrano eventi di diverso tipo, come `kind:0, 1, 7, 30023, 9734, 9735, 9736`. Grazie ai relay, i messaggi e i micropagamenti vengono propagati in modo affidabile e ridondante, garantendo la continuità delle comunicazioni anche in caso di malfunzionamento di un nodo.

I **nodi Lightning** costituiscono la parte economica del sistema. Sono i componenti che creano e gestiscono le fatture necessarie per sbloccare i contenuti protetti. Ogni nodo è configurato nel file `nodes-config.json`, che contiene le informazioni necessarie per la connessione (host, certificato TLS e *macaroon* per l'autenticazione). La comunicazione con i nodi Lightning avviene in modo controllato: il client non gestisce mai direttamente i fondi dell'utente. Questo approccio segue la filosofia *non-custodial*, preferita per la maggiore sicurezza e trasparenza, ma sono stati effettuati anche test con soluzioni *custodial* (come descritto nel Capitolo 5), per valutarne la praticità d'uso.

Il collegamento tra il frontend e l'infrastruttura Lightning è gestito da un piccolo modulo di **API backend**, anch'esso sviluppato in *Next.js*. Le API espongono endpoint specifici, come `/api/create-invoice` e `/api/pay-invoice`, che agiscono da ponte sicuro verso LND. In questo modo, operazioni delicate come la generazione o il pagamento delle fatture vengono gestite da un livello intermedio, che protegge le chiavi dei nodi e riduce il rischio di accessi diretti non autorizzati.

Infine, lo **storage locale** fornisce un semplice sistema di salvataggio dei dati. Le informazioni relative agli *Zap*, ai contenuti sbloccati e alle preferenze dell'utente vengono memorizzate direttamente nel browser, attraverso `localStorage` e `sessionStorage`. Questo approccio evita l'uso di database centralizzati, in linea con la natura decentralizzata del progetto.

Nel complesso, l'architettura di *ZapGate* mostra come sia possibile realizzare un sistema coerente con i principi del Web decentralizzato, unendo la velocità della rete Lightning alla flessibilità del protocollo Nostr. Il ri-

sultato è una piattaforma leggera, sicura e facilmente estendibile, capace di dimostrare la fattibilità di un modello di micropagamento distribuito e trasparente.

8.4 Testing del prototipo

Dopo la fase di sviluppo, il prototipo di *ZapGate* è stato testato al fine di verificarne il comportamento in un contesto d'uso realistico e di osservare l'effettiva integrazione tra le componenti Nostr e Lightning Network. Una volta effettuato l'accesso tramite *nos2x*, l'utente dispone di un ambiente operativo completo che riproduce le funzionalità tipiche di un social network moderno. In particolare, è possibile scrivere nuovi post, leggere e acquistare contenuti pubblicati da altri utenti, accedere alla sezione dedicata ai *creator* seguiti e a quella dei *trend*, che mostra i post con il maggior numero di interazioni. È inoltre presente un'area informativa che raccoglie la documentazione relativa alle specifiche Nostr — tra cui *ZapRequest* e *ZapReceipt* — e una *dashboard* personale che consente di modificare i propri metadati e di consultare grafici (*charts*) sulle vendite dei propri contenuti.

Per simulare un utilizzo reale, è stato configurato un ambiente di test basato su tre nodi *Polar*, descritti nel capitolo dedicato alla sperimentazione. In questa configurazione, un primo utente ha pubblicato un post protetto da pagamento, mentre un secondo utente ha proceduto all'acquisto dello stesso. Al completamento della transazione Lightning — processata tramite *Polar* — il contenuto è risultato immediatamente sbloccato, confermando la corretta esecuzione del flusso di pagamento e la sincronizzazione tra client e relay.

Questa interazione riproduce in modo fedele uno degli scenari d'uso più rilevanti per il modello *ZapGate*, ovvero quello legato all'editoria digitale e alla vendita diretta di contenuti. L'esperimento ha mostrato la rapidità dell'intero processo, con tempi di conferma praticamente istantanei sia nei test condotti in ambiente locale con *Polar*, sia nelle simulazioni che hanno coinvolto soluzioni *custodial* come *Alby*. La velocità nella registrazione delle microtransazioni rappresenta uno degli aspetti più promettenti del sistema, insieme all'assenza di costi di commissione per l'utente finale.

Dai test è emerso inoltre come le prestazioni complessive dipendano in larga misura dalla qualità e affidabilità dei relay coinvolti: essendo *ZapGate* un sistema relay-dipendente, la stabilità e la capacità di gestione dei nodi influiscono direttamente sull'esperienza d'uso e sulla fluidità delle interazioni. In generale, tuttavia, i risultati confermano la fattibilità tecnica e operativa del modello proposto, evidenziando la possibilità concreta di implementare un meccanismo di micropagamento nativo e trasparente all'interno dell'ecosistema Nostr.

8.5 Implementazione tecnica

Dal punto di vista tecnico, l'implementazione di *ZapGate* si basa su una serie di librerie interne che coordinano le principali funzionalità del sistema, garantendo l'integrazione tra il protocollo Nostr e la rete Lightning. Queste librerie sono organizzate in moduli distinti, ciascuno con responsabilità ben definite, in modo da favorire la manutenzione e la riusabilità del codice.

La libreria `lib/nostr.ts` costituisce il cuore della comunicazione con la rete Nostr. Al suo interno sono implementate le funzioni per la creazione, la firma e la pubblicazione degli eventi, seguendo le specifiche del protocollo. Include inoltre generatori dedicati per la gestione di diversi tipi di evento, come gli *Zap*, i commenti e le reazioni, permettendo di gestire in modo uniforme l'intero flusso di interazione tra utenti e relay.

La libreria `lib/zaps.ts` si occupa invece del ciclo completo dei micropagamenti. A partire dalla generazione della fattura Lightning, gestisce tutte le fasi successive: la richiesta di pagamento, la verifica dell'avvenuta transazione e la pubblicazione della relativa ricevuta su Nostr. Questo modulo rappresenta il punto di integrazione tra la logica economica e quella comunicativa del sistema, consentendo di tradurre un'operazione finanziaria in un evento verificabile e condivisibile all'interno della rete.

L'interfaccia utente è stata progettata con particolare attenzione alla modularità e alla chiarezza visiva. È composta da componenti riutilizzabili che coprono le principali sezioni dell'applicazione: le schede dei contenuti, le aree dedicate ai commenti, i pulsanti di reazione e di invio *Zap*, i grafici che mostrano i dati sulle vendite e la barra di navigazione principale, dotata anche di una modalità scura (*dark mode*). Questa struttura a componenti semplifica lo sviluppo di nuove funzionalità e assicura un'esperienza coerente per l'utente finale.

8.6 Sicurezza e configurazione

La sicurezza rappresenta uno degli aspetti più rilevanti nella progettazione di *ZapGate*, poiché l'applicazione integra componenti che operano su due livelli delicati: la gestione di micropagamenti sulla rete Lightning e la conservazione delle chiavi crittografiche degli utenti. Per questo motivo, sono state adottate diverse misure atte a garantire la protezione dei dati e la sicurezza delle transazioni, senza compromettere la natura decentralizzata del sistema.

L'accesso ai nodi Lightning avviene esclusivamente tramite meccanismi di autenticazione sicuri, basati su *macaroon* e certificati *TLS*. In questo modo si assicura che solo client autorizzati possano interagire con l'infrastruttura di pagamento, riducendo il rischio di connessioni non verificate o manomissioni.

Per prevenire l'esposizione diretta dei nodi, è stato introdotto un **backend proxy** che funge da intermediario tra il frontend e LND. Questo livello aggiuntivo permette di filtrare le richieste e di gestire le operazioni sensibili, come la creazione o il pagamento delle fatture, senza rivelare le credenziali o gli endpoint reali dei nodi Lightning.

Dal lato utente, la sicurezza delle chiavi private è garantita attraverso l'uso delle estensioni compatibili con lo standard **NIP-07**. In questo modo, la firma degli eventi avviene direttamente nel browser, mantenendo le chiavi sempre sotto il pieno controllo dell'utente e impedendo che vengano mai trasmesse a servizi esterni.

Le variabili sensibili, come le API key o i percorsi di connessione ai nodi, vengono gestite tramite file di configurazione `.env`, che non sono inclusi nel codice sorgente pubblico. Questa accortezza previene la divulgazione accidentale di informazioni riservate e facilita la gestione di ambienti di sviluppo e produzione separati.

Infine, i dati relativi all'utente e alle sue attività sono conservati esclusivamente in locale, utilizzando `localStorage` e `sessionStorage`. Non vengono impiegati database centralizzati né meccanismi di sincronizzazione remota, in modo da ridurre al minimo la superficie di attacco e preservare la privacy dell'utente.

8.7 Limitazioni del prototipo

Nonostante i risultati ottenuti siano complessivamente positivi e abbiano confermato la fattibilità tecnica del modello proposto, il prototipo di *ZapGate* presenta ancora alcune limitazioni, dovute principalmente alla natura sperimentale del progetto e all'ambiente di test in cui è stato sviluppato.

In primo luogo, durante le fasi di sperimentazione in locale è stato necessario disabilitare la verifica automatica dei certificati SSL, impostando la variabile `NODE_TLS_REJECT_UNAUTHORIZED=0`. Questa soluzione, adottata unicamente per semplificare la comunicazione tra i componenti del sistema in ambiente di sviluppo, non è tuttavia adatta a un contesto di produzione, poiché riduce la protezione contro possibili attacchi di tipo *man-in-the-middle*.

La piattaforma dipende inoltre in modo significativo dalla disponibilità e dall'affidabilità dei **relay Nostr**. Poiché la sincronizzazione dei contenuti e la propagazione degli eventi avvengono attraverso di essi, eventuali rallentamenti o interruzioni nei relay possono influenzare direttamente l'esperienza dell'utente e la coerenza dei dati tra diversi client.

Infine, la gestione dei micropagamenti è stata testata esclusivamente in un ambiente controllato, utilizzando una configurazione basata su *Polar*. Questo approccio ha permesso di verificare il corretto funzionamento delle

transazioni senza coinvolgere fondi reali, ma non consente ancora di valutare appieno le prestazioni e la scalabilità del sistema in scenari di utilizzo reale.

In prospettiva, l'evoluzione del progetto dovrà quindi affrontare questi limiti, introducendo meccanismi di sicurezza più avanzati, relay più performanti e un'integrazione con nodi Lightning operativi in rete pubblica, così da avvicinarsi sempre di più a una versione pienamente produttiva del sistema.

8.8 Validità della demo come proof-of-concept

Il prototipo di *ZapGate* costituisce una dimostrazione concreta e convincente della fattibilità del modello proposto. Pur essendo una versione sperimentale, la piattaforma riesce a integrare in modo efficace gli aspetti tecnici e concettuali alla base del progetto, offrendo una prova tangibile della possibilità di costruire un ecosistema sociale ed economico distribuito.

Dal lato **Lightning**, la demo mostra l'intero ciclo di una transazione: dalla creazione della fattura alla sua esecuzione, fino alla registrazione dell'avvenuto pagamento all'interno della rete Nostr. Questo processo permette di verificare come i micropagamenti possano essere gestiti in maniera automatica, sicura e trasparente, senza la necessità di intermediari o infrastrutture centralizzate.

Dal lato **Nostr**, il sistema utilizza diversi tipi di eventi standard (tra cui `kind:0`, `kind:1`, `kind:7`, `kind:9734`, `kind:9735` e `kind:9736`), dimostrando la flessibilità del protocollo nel gestire contenuti, profili, interazioni sociali e flussi economici. La capacità di combinare in un unico ambiente la pubblicazione di contenuti e la loro monetizzazione rappresenta uno degli aspetti più innovativi della piattaforma.

In conclusione, *ZapGate* si configura come una **proof-of-concept** pienamente valida: pur non essendo ancora destinata alla produzione, essa conferma la solidità del modello di riferimento e la compatibilità tra Nostr e Lightning Network. Il progetto dimostra quindi che è tecnicamente possibile costruire un sistema sociale decentralizzato in cui la distribuzione dei contenuti e la remunerazione dei creator avvengono in modo diretto, trasparente e verificabile attraverso tecnologie aperte.

9 Analisi economica del modello Zap nell'editoria digitale: il caso ZapGate

Il tema dei micropagamenti accompagna l'economia digitale da anni, ma finora è rimasto più una promessa che una realtà. Le tecnologie tradizionali, infatti, non consentono di gestire in maniera sostenibile transazioni di importo molto ridotto. Servizi consolidati come PayPal, Stripe o le carte di credito applicano commissioni pari in media al 2,9% più 0,35 € a transazione. Questo modello funziona per importi medio-alti, ma rende di fatto impraticabili i pagamenti di pochi centesimi.

Un esempio concreto lo rende evidente: se un lettore volesse pagare 0,05 € per leggere un articolo online, la sola commissione fissa (circa 0,35 €) supererebbe di gran lunga l'importo stesso, annullando la convenienza. Neppure i bonifici SEPA rappresentano una soluzione valida: sebbene economici, richiedono in media 1–2 giorni di elaborazione e risultano quindi inadatti a transazioni rapide, granulari e di basso valore.

9.1 Lightning Network come abilitatore dei micropagamenti

Con l'avvento di Lightning Network (LN) lo scenario cambia radicalmente. La commissione media per una transazione si aggira intorno a 1 satoshi (circa 0,00025 €), un importo talmente ridotto da risultare irrilevante anche per micropagamenti di pochi centesimi. LN abilita quindi ciò che prima era impossibile: transazioni globali, istantanee e con costi trascurabili, applicabili non solo agli acquisti digitali tradizionali ma anche a flussi continui di micro-ricavi.

La differenza è evidente in un confronto pratico:

- con carta di credito o PayPal, un pagamento di 0,50 € (2000 sats) può generare oltre il 50% di commissioni;
- con Lightning, la fee resta pari a 1 sat, cioè praticamente zero.

9.2 Simulazione su larga scala

Per valutare l'impatto economico che gli *zap* potrebbero avere, è stata condotta una simulazione Monte Carlo su un milione di transazioni. Questa tecnica si basa sulla generazione di numeri casuali e sulla loro analisi statistica, permettendo di stimare l'andamento complessivo di un sistema complesso. Nel nostro caso è stata utilizzata una distribuzione lognormale, che rappresenta in modo realistico la dinamica attesa: molti micropagamenti di importo molto ridotto e pochi pagamenti di valore più consistente.

I risultati sono chiari. Con i sistemi di pagamento tradizionali, le commissioni complessive superano i 350.000 €. Con Lightning, invece, i costi si riducono a poche centinaia di euro. Anche ipotizzando un piccolo costo di servizio a favore di provider come relay, nodi o piattaforme, la differenza rimane enorme: i costi restano inferiori di due ordini di grandezza rispetto ai metodi tradizionali.

Un esempio concreto rende ancora più evidente il vantaggio: applicando una commissione simbolica dello 0,1% per ogni transazione, le fee totali ammontano a circa 3.500 €, contro i 350.000 € richiesti dalle infrastrutture tradizionali. Questo dimostra come Lightning non sia soltanto una soluzione più economica, ma anche un abilitatore di modelli di business che sarebbero del tutto insostenibili nell'economia fiat.

Nota bene: i dati non derivano da osservazioni dirette della rete Lightning. Gli importi sono stati generati secondo una distribuzione lognormale ($\text{mean} = 4.5$, $\sigma = 0.8$), con esclusione degli outlier superiori a 100.000 sats. La scelta della distribuzione riflette il comportamento tipico dei micropagamenti: molti trasferimenti di importo ridotto e pochi di valore maggiore.

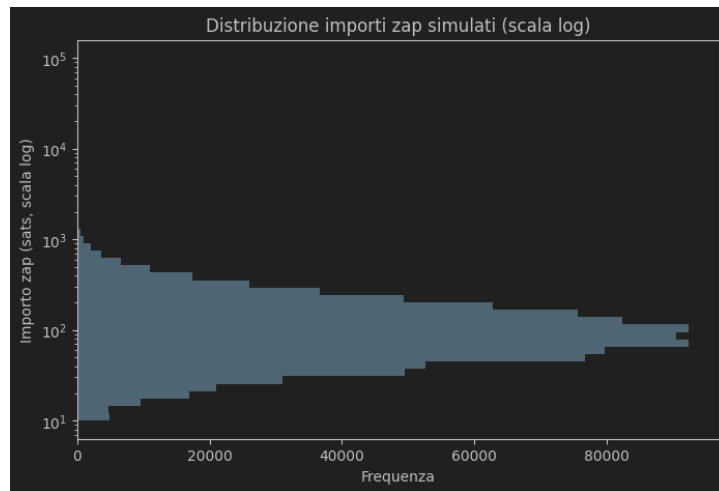


Figura 9.1: Distribuzione degli importi degli zap simulati (scala logaritmica). La maggior parte delle transazioni ha importi molto bassi, mentre pochi zap hanno importi più elevati: questo rispecchia la realtà dei micropagamenti online.

9.3 Il ruolo del provider

Un aspetto cruciale riguarda la sostenibilità economica dei provider della rete: relay, nodi e operatori dell'infrastruttura. Se le fee di rete sono così basse, come possono mantenersi nel lungo periodo?

Due modelli alternativi appaiono possibili:

- una **flat fee minima**, ad esempio 1 satoshi per ogni zap, aggiunta al costo di rete;
- una **commissione percentuale ridotta**, ad esempio lo 0,1%, trascurabile per l'utente ma significativa a grandi volumi.

La simulazione dimostra che, anche introducendo una commissione minima, su un milione di zap i provider ottengono ricavi sufficienti senza compromettere la convenienza per utenti e creatori. Lightning Network concilia quindi sostenibilità economica e scalabilità.

9.4 Proiezioni e impatto

Immaginando un ecosistema con un milione di utenti attivi che inviano dieci zap al giorno da 100 sats ciascuno, su base annua si otterrebbero oltre 365 miliardi di sats in circolazione, equivalenti a circa 90.000 BTC di volume.

In tale scenario:

- le fee della rete LN si ridurrebbero a poche migliaia di euro;
- i sistemi tradizionali risulterebbero del tutto ingestibili, a causa dei costi fissi sproporzionati.

La differenza non è solo quantitativa ma anche qualitativa: Lightning apre la strada a un'economia nativa di Internet, caratterizzata da pagamenti granulari, frequenti, trasparenti e proporzionati al valore effettivamente consumato.

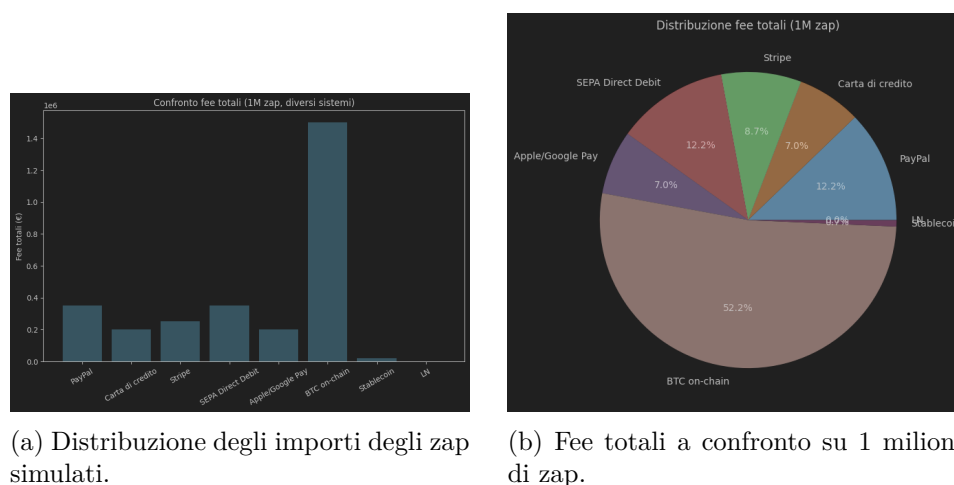


Figura 9.2: Analisi simulata degli zap: (a) distribuzione degli importi, (b) confronto delle fee totali tra metodi tradizionali e Lightning.

9.5 Applicazione all'editoria digitale

Il settore editoriale online è un ottimo esempio per comprendere il potenziale del modello *Zap*. Oggi, infatti, la maggior parte degli editori si affida a due strategie principali di monetizzazione. La prima è quella degli **abbonamenti mensili**, che obbligano l'utente a un pagamento fisso indipendentemente dal consumo effettivo. La seconda è la **pubblicità online**, che genera ricavi variabili e spesso dipende dal controllo esercitato da grandi piattaforme centralizzate.

Il modello *Zap*, sfruttando Lightning Network, introduce un'alternativa molto più flessibile: consente la monetizzazione dei singoli contenuti, con costi di transazione minimi e senza intermediari. Applicazioni come **ZapGate** dimostrano già la fattibilità concreta di questa strategia.

9.6 Modelli di monetizzazione

All'interno dell'editoria digitale, il modello *Zap* può assumere diverse forme operative:

- **Pay-per-article:** l'utente paga una cifra molto ridotta (tra 10 e 200 satoshi) per accedere a un singolo contenuto;
- **Tip-based:** i contenuti sono accessibili gratuitamente, ma gli utenti possono inviare *zap* volontari come forma di supporto all'autore;
- **Modello ibrido:** una parte dei contenuti rimane gratuita, mentre articoli di approfondimento o materiali premium vengono sbloccati solo tramite *zap*.

Il ricavo mensile per un editore può essere stimato con la formula:

$$R = U \times Z \times V$$

dove:

- U è il numero di utenti attivi,
- Z è il numero medio di *zap* inviati da ciascun utente al mese,
- V è il valore medio di ciascuno *zap*.

9.7 Scenari di applicazione

Per valutare la sostenibilità del modello *Zap* nell'editoria, sono stati ipotizzati tre scenari progressivi. Essi differiscono per il numero di utenti coinvolti, la frequenza degli *zap* e il valore medio delle transazioni. In questo modo si copre l'intero spettro dei possibili casi d'uso: dal giornalista indipendente che monetizza articoli singoli, fino a grandi editori mainstream che possono integrare gli *zap* come fonte strutturale di ricavi.

Scenario A – Giornalista indipendente

Parametro	Valore
Utenti attivi	~1.000
Zap medi/mese	5
Valore medio zap	50 sats (0,02 €)
Ricavo stimato	~100 €/mese

Tabella 9.2: Scenario A – Giornalista indipendente.

In questo caso, gli zap si configurano come una forma di *crowdfunding distribuito*, utile ad affiancare altre fonti di reddito.

Scenario B – Piccola rivista online

Parametro	Valore
Utenti attivi	~10.000
Zap medi/mese	10
Valore medio zap	100 sats (0,04 €)
Ricavo stimato	~4.000 €/mese

Tabella 9.4: Scenario B – Piccola rivista online.

In questo contesto, il modello Zap diventa competitivo con piccoli abbonamenti o con i ricavi pubblicitari, riuscendo a includere anche i lettori occasionali.

Scenario C – Editore mainstream

Parametro	Valore
Utenti attivi	~100.000
Zap medi/mese	20
Valore medio zap	200 sats (0,08 €)
Ricavo stimato	~160.000 €/mese

Tabella 9.6: Scenario C – Editore mainstream.

In questo scenario, il modello Zap può competere direttamente con i ricavi pubblicitari tradizionali, fornendo un'alternativa più trasparente e diretta.

Tabella comparativa

Per completezza, nella Tabella 9.8 sono riportati insieme i tre scenari, suddivisi in due blocchi verticali per migliorare la leggibilità.

Scenario	Utenti	Zap/mese	Valore (sats)	Ricavo (€)
A – Giornalista	1.000	5	50	~100
B – Rivista online	10.000	10	100	~4.000
C – Editore main.	100.000	20	200	~160.000

Tabella 9.8: Confronto sintetico tra i tre scenari di adozione del modello Zap.

9.8 Complementarità con il modello ad abbonamento

Gli *zap* garantiscono una notevole flessibilità, ma non assicurano stabilità economica nel lungo periodo. Per questo motivo, possono essere affiancati da modelli di abbonamento basati su Lightning, capaci di offrire una maggiore prevedibilità dei ricavi. Le forme possibili includono:

- **Pagamenti ricorrenti LN**, con addebito periodico in satoshi;
- **ZapPool**, ovvero crediti mensili redistribuiti sui contenuti effettivamente consumati;
- **Relay premium**, riservati esclusivamente agli abbonati;
- **Servizi extra**, come feed personalizzati o notifiche prioritarie.

Gli abbonamenti risultano particolarmente attrattivi per diverse categorie di utenti:

1. **Lettori intensivi**, che con i micropagamenti rischierebbero di sostenere spese maggiori rispetto a una quota fissa;
2. **Utenti interessati a contenuti esclusivi**, come report o podcast premium;
3. **Chi privilegia la comodità** di un pagamento unico e ricorrente;

4. Chi desidera sostenere continuativamente un progetto, in forma simile al crowdfunding.

Gli *zap* e gli abbonamenti non devono quindi essere visti come modelli concorrenti, ma piuttosto come complementari: i primi sono ideali per gli utenti occasionali e per monetizzare singoli contenuti, mentre i secondi garantiscono stabilità agli editori e convenienza ai lettori abituali.

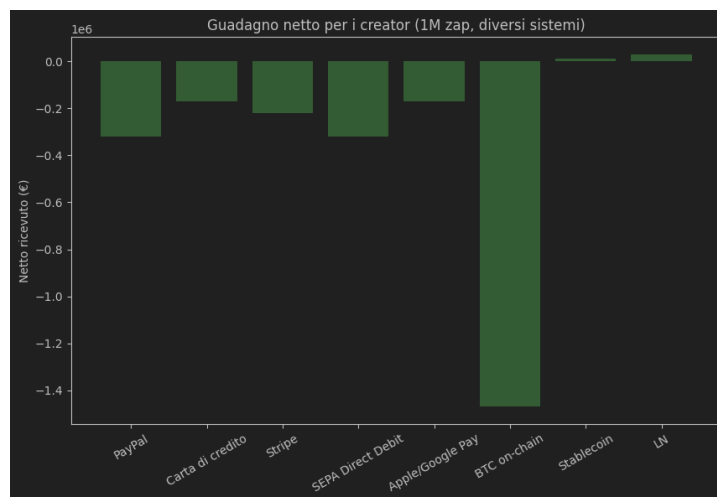


Figura 9.3: Guadagno netto per i creator: LN preserva quasi intatto il valore inviato dagli utenti, a differenza dei sistemi tradizionali che erodono i micropagamenti con fee sproporzionate.

9.9 Distribuzione dei ricavi e legge di Pareto

Come accade in molte piattaforme digitali, anche nel caso degli *zap* si manifesta il cosiddetto effetto Pareto: una piccola percentuale di utenti è responsabile della maggior parte delle entrate complessive. In pratica, pochi utenti particolarmente attivi o generosi tendono a sostenere in misura significativa l'ecosistema, mentre la maggioranza contribuisce in maniera più limitata.

Questo fenomeno non rappresenta necessariamente un problema: al contrario, è una caratteristica ricorrente nei modelli economici online e deve essere considerata nella progettazione delle strategie di monetizzazione. Per esempio, sapendo che una parte ristretta degli utenti genera la quota principale dei ricavi, un editore può strutturare offerte o servizi premium mirati proprio a questa fascia, massimizzando l'impatto economico complessivo. Si distinguono due categorie:

- **Casual user** (80%): inviano 1 *zap* a settimana (≈ 200 sats/mese);
- **Heavy user** (20%): inviano 5 *zap* al giorno (≈ 7.500 sats/mese).

Tipo utente	% utenti	Zap/mese	Valore medio (sats)
Casual	80%	4	50
Heavy	20%	150	50
Totale	100%	–	–

Tipo utente	Spesa media (sats)	Ricavo totale (€)
Casual	200	~160
Heavy	7.500	~1.200
Totale	–	~1.360

Tabella 9.11: Distribuzione dei ricavi tra utenti casual e heavy, suddivisa in due blocchi.

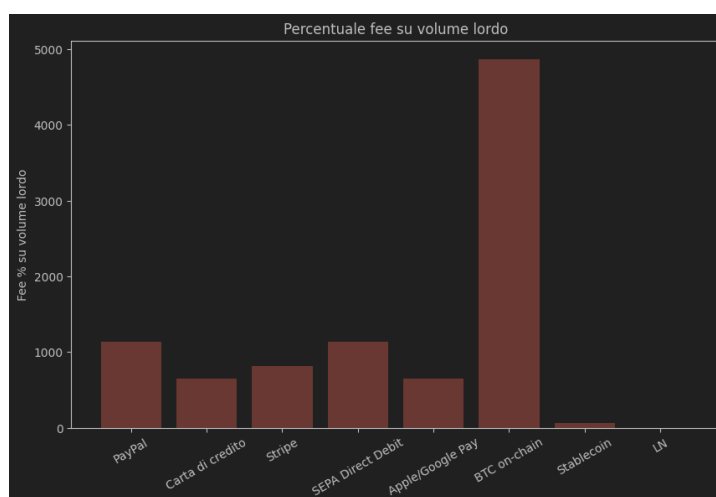


Figura 9.4: Percentuale fee rispetto al volume lordo: PayPal/Stripe arrivano facilmente al 10, LN rimane sempre sotto l'1

9.10 Punti di forza e criticità del modello Zap

Il modello *Zap* presenta una serie di vantaggi chiari e facilmente percepibili. Le transazioni avvengono in modo pressoché istantaneo e con fee praticamente nulle, il che consente di monetizzare anche contenuti minori che nei sistemi tradizionali non avrebbero alcuna sostenibilità economica. Si tratta inoltre di un modello inclusivo: anche piccoli importi, se diffusi su larga scala, diventano significativi per i creatori. Un ulteriore punto di forza è la trasparenza: i flussi economici sono pubblici e verificabili attraverso Nostr e Lightning Network. Infine, *Zap* mostra una grande elasticità, adattandosi

sia a giornalisti indipendenti che a grandi editori, senza imporre un modello unico di fruizione.

Accanto a questi vantaggi, non mancano alcune criticità. In primo luogo, i ricavi generati da *Zap* sono variabili e difficili da prevedere con certezza, a differenza dei modelli basati su abbonamento. In secondo luogo, la necessità di utilizzare un wallet Lightning rappresenta una barriera tecnologica che non tutti gli utenti sono disposti o in grado di superare. Va poi considerata la diffusa cultura del “gratis” che caratterizza l'informazione online: molti utenti non sono abituati a pagare per i contenuti digitali, il che può ridurre la propensione a utilizzare micropagamenti. Infine, al momento mancano strumenti di fidelizzazione consolidati, come sistemi di *loyalty* o programmi di premi, che potrebbero incentivare la continuità dei contributi.

Modello	Stabilità ricavi	Barriera ingresso	Costi di intermediazione
Abbonamento	Alta	Alta	Medio/Alto
Pubblicità	Media/Bassa	Nulla	Alto
Zap	Variabile	Bassa	Molto bassa

Modello	Adatto a occasionali	Trasparenza
Abbonamento	No	Media
Pubblicità	Sì, ma intrusiva	Bassa
Zap	Sì	Alta

Tabella 9.14: Confronto tra modelli di monetizzazione digitale.

9.11 Prospettive e lavori correlati

I micropagamenti nell'economia tradizionale sono impraticabili per i costi fissi; con Lightning diventano sostenibili e scalabili. Questo non è solo un guadagno in efficienza, ma un cambio di paradigma: modelli come *pay-per-article*, *streaming money* e *machine-to-machine* diventano realizzabili.

ZapGate rappresenta un esempio concreto di questo potenziale, dimostrando come i micropagamenti possano trasformare l'economia digitale e restituire potere a consumatori e produttori di contenuti.

Rispetto ai client Nostr esistenti, che integrano gli zap in modo eterogeneo, ZapGate si distingue per:

- una pipeline esplicita 9734→LNURL→BOLT11→9735;
- metriche di esperienza utente e affidabilità misurate;

- policy di sicurezza (pin dei relay, finestra `created_at`, TLS, deleghe NIP-26);
- replicabilità, grazie a dataset e script disponibili.

Il contributo non è un nuovo wallet o relay, ma un collante dimostrativo che massimizza interoperabilità e verificabilità end-to-end.

Parte III

Considerazioni Finali

10 Conclusioni

In questo lavoro abbiamo indagato la possibilità di abilitare micropagamenti verificabili nell’ecosistema Nostr, proponendo e realizzando il prototipo **ZapGate**. Le conclusioni possono essere riassunte in quattro direttrici principali: risultati ottenuti, trade-off osservati, limiti sperimentali e prospettive future.

Risultati dimostrati

Con ZapGate abbiamo mostrato che l’integrazione tra **NIP-57 (9734/9735)** e la famiglia **LNURL (LUD-06/LUD-16)** consente di realizzare un flusso end-to-end coerente di micropagamenti:

$$9734 \rightarrow LNURL \rightarrow BOLT11 \rightarrow 9735.$$

Il prototipo ha dimostrato che la ricevuta 9735 può essere utilizzata per *sbloccare contenuti*, alimentare *metriche di reputazione* e supportare percorsi sia *custodial* sia *non-custodial*. Sono state anche esplicitate minacce concrete e possibili contromisure (selezione dei relay, deduplicazione, deleghe firmate, policy TLS), chiarendo come garantire **autenticità**, **integrità** e **verificabilità pubblica**.

Trade-off custodial vs non-custodial

L’analisi ha evidenziato differenze sostanziali tra i due approcci:

- Il percorso **custodial** riduce al minimo l’attrito per l’utente, semplificando l’onboarding (canali e liquidità già disponibili) e migliorando il *time-to-first-zap*. Tuttavia introduce dipendenza da terze parti, rischi di sorveglianza e minore trasparenza.
- Il percorso **non-custodial** garantisce piena sovranità e privacy operativa, ma richiede setup tecnico (nodi, canali, gestione della liquidità) e tempi on-chain iniziali.

La scelta, pertanto, è **situazionale**: per utenti alle prime armi può essere opportuno iniziare con un approccio custodial e successivamente migrare; per creator e operatori orientati alla resilienza, è preferibile il modello non-custodial.

Limiti sperimentali

Le valutazioni sono state condotte in **ambienti di test** (regtest/testnet), con un **campione ridotto** di wallet e relay. Non disponiamo quindi di una copertura esaustiva sulla compatibilità tra client, né di dati relativi alla latency su Internet pubblica. Alcuni compromessi adottati (ad esempio `NODE_TLS_REJECT_UNAUTHORIZED=0` in locale) sono accettabili solo in fase di dimostrazione, ma non in produzione. Tali limiti non invalidano la **validazione funzionale** del flusso, ma vanno tenuti presenti per un'eventuale estensione.

Prospettive future

Il lavoro apre a tre piste di sviluppo rilevanti:

1. **Deleghe granulari (NIP-26)**: permettere la firma di ricevute 9735 da parte di bot delegati, con vincoli temporali e per-kind, semplificando audit e riducendo i rischi di gestione delle chiavi.
2. **NWC / NIP-47 (wallet-connect)**: introdurre un'esperienza utente più fluida, integrando client e wallet (anche mobile) senza sacrificare la separazione dei privilegi.
3. **Bridging relay**: implementare repliche selettive di eventi 9734/9735 con policy di deduplicazione, per aumentare la visibilità e ridurre i casi in cui la richiesta e la ricevuta viaggiano su relay diversi.

Linee guida operative

A valle del prototipo, si possono già proporre alcune raccomandazioni pratiche per implementazioni reali:

- **Pin dei relay** nei tag della ZapRequest e interrogazione mirata della ZapReceipt (9735).
- **Validità temporale ridotta di created_at** (± 5 minuti) e utilizzo di tag di deduplica (`zap_id`) per mitigare spam e duplicazioni.
- **TLS obbligatorio** sugli endpoint LNURL, rifiutando `http://` e verificando la coerenza tra dominio e `lud16`.

- **Deleghe NIP-26** per i bot che pubblicano eventi 9735, con scope limitato (kind, scadenza, rate-limit).
- **Percorso ibrido:** prevedere un onboarding custodial iniziale con migrazione assistita verso non-custodial, tramite backup ed export delle chiavi.

In sintesi, il lavoro dimostra che la combinazione tra Nostr e Lightning abilita un modello di micropagamenti decentralizzato e verificabile, capace di coniugare efficienza tecnica e nuove opportunità economiche. Pur con i limiti delle simulazioni, i risultati offrono una base solida per sviluppi futuri e per la diffusione di modelli di business più equi e sostenibili.

Parte IV

Bibliografia e Appendici

Bibliografia

- [1] Fidelity Digital Assets, “The Lightning Network: Expanding Bitcoin Use Cases,” 2023. indirizzo: https://www.fidelitydigitalassets.com/sites/g/files/djuvja3256/files/acquiadam/FDA_TheLightningNetwork_ExpandingBitcoinUseCases_1187503.1.0_V5.pdf.
- [2] Nostr Community. “Nostr Zap Latency Discussion.” (2023), indirizzo: <https://nostr.com/nprofile1qyt8wumn8ghj7un9d3shjtnddaehgu3wvp6kytc...>
- [3] Pobble Labs. “Nostr Relay Performance Benchmarks.” (2023), indirizzo: https://code.pobblelabs.org/nostr_relay/doc/tip/docs/performance.md.
- [4] Voltage. “How many transactions can the Lightning Network handle?” (2023), indirizzo: <https://www.voltage.cloud/blog/how-many-transactions-can-the-lightning-network-handle>.
- [5] Lightning Network. “BOLT #11: Invoice encoding.” Formato fatture LN. (2025), indirizzo: <https://github.com/lightning/bolts/blob/master/11-payment-encoding.md>.
- [6] Nostr Protocol. “NIP-01: Basic protocol flow.” Struttura eventi e flusso base. (2025), indirizzo: <https://github.com/nostr-protocol/nips/blob/master/01.md>.
- [7] Nostr Protocol. “NIP-07: window.nostr (browser).” Firma eventi via estensioni. (2025), indirizzo: <https://nips.nostr.com/7>.
- [8] Nostr Protocol. “NIP-11: Relay Information Document.” Metadati e capability dei relay. (2025), indirizzo: <https://github.com/nostr-protocol/nips/blob/master/11.md>.
- [9] Nostr Protocol. “NIP-26: Delegated Event Signing.” Delega sicura a bot/servizi. (2025), indirizzo: <https://github.com/nostr-protocol/nips/blob/master/26.md>.

- [10] Nostr Protocol. “NIP-47: Nostr Wallet Connect (NWC).” API di collegamento app wallet LN. (2025), indirizzo: <https://nips.nostr.com/47>.
- [11] Nostr Protocol. “NIP-57: Lightning Zaps.” Definisce eventi 9734/9735. (2025), indirizzo: <https://github.com/nostr-protocol/nips/blob/master/57.md>.
- [12] Nostr Protocol. “NIPs (standard Nostr).” Raccolta ufficiale delle specifiche e stato. (2025), indirizzo: <https://github.com/nostr-protocol/nips>.
- [13] Alby. “@getalby/sdk.” SDK per integrazione NWC. (), indirizzo: <https://www.npmjs.com/package/@getalby/sdk>.
- [14] Alby. “Alby (estensione browser).” WebLN/NIP-07 e NWC. (), indirizzo: <https://guides.getalby.com/>.
- [15] Bitcoin Developers. “Bitcoin Core: regtest & RPC.” Script e comandi per test locali. (), indirizzo: <https://developer.bitcoin.org/examples/testing.html>.
- [16] Core Lightning. “CLN: lightningd & cli.” Alternativa a LND. (), indirizzo: <https://docs.corelightning.org/reference/lightning-cli>.
- [17] Damus Project. “Damus (client iOS/macOS).” Esempio di client con supporto zaps. (), indirizzo: <https://github.com/damus-io/damus>.
- [18] J. J. “Polar.” Reti LN locali/regtest (GUI). (), indirizzo: <https://github.com/jamaljsr/polar>.
- [19] Lightning Labs. “LND: Incli & API.” Riferimenti ufficiali API/CLI. (), indirizzo: <https://lightning.engineering/api-docs/api/lnd/>.
- [20] Lightning Network. “BOLTs (spec Lightning).” Raccolta ufficiale delle BOLT. (), indirizzo: <https://github.com/lightning/bolts>.
- [21] LNbits Project. “LNbits.” Servizio custodial usato in demo (wallet e API). (), indirizzo: <https://docs.lnbits.org/>.
- [22] LNURL Project. “LNURL LUDs.” Specifiche LNURL modulari. (), indirizzo: <https://github.com/lnurl/luds>.
- [23] LNURL Project. “LUD-16: Lightning Address.” Alias email-like per i pagamenti. (), indirizzo: <https://github.com/lnurl/luds>.
- [24] Meta. “React.” Framework UI. (), indirizzo: <https://react.dev/>.
- [25] Tailwind Labs. “Tailwind CSS.” Utility-first CSS. (), indirizzo: <https://tailwindcss.com/docs/installation>.

-
- [26] Vercel. “Next.js.” App router e API routes. (), indirizzo: <https://nextjs.org/>.
 - [27] Zeus Team. “Zeus (wallet).” Embedded LND e NWC. (), indirizzo: <https://docs.zeusln.app/for-users/using-zeus/wallets/>.

Appendice A – Setup e Replicabilità

A.1 Versioni consigliate

Per garantire la replicabilità degli esperimenti, si riportano le versioni degli strumenti utilizzati:

- **Polar:** 2.0.x
- **LND:** 0.17.x-beta (regtest)
- **Bitcoin Core:** 26.x (regtest)
- **Node.js:** 20.x LTS
- **Next.js:** 14.x
- **nostr-tools:** 1.17.x
- **@getalby/sdk:** ultima versione compatibile (NWC)

A.2 Esempio di configurazione `nodes-config.json`

Il seguente file JSON mostra un esempio di configurazione minima per collegare nodi Lightning gestiti tramite Polar, includendo i percorsi dei certificati TLS e dei macaroon necessari per l'autenticazione.

```
{
  "nodes": [
    {
      "host": "https://127.0.0.1:8080",
      "tlsCertPath": "./certs/alice/tls.cert",
      "macaroonHex": "<MACAROON_ADMIN_HEX>",
      "nostr_pubkey": "<ALICE_PUBKEY>"
    },
    {
      "host": "https://127.0.0.1:8081",
      "tlsCertPath": "./certs/bob/tls.cert",
      "macaroonHex": "<MACAROON_INVOICE_HEX>",
      "nostr_pubkey": "<BOB_PUBKEY>"
    }
  ]
}
```

Appendice B – Ulteriori Materiali

B.1 Script e dataset

Gli script usati per la simulazione e i dataset generati sono resi disponibili su repository GitHub (link omissso in questa copia). Essi comprendono:

- script di generazione di micropagamenti simulati con distribuzione lognormale (Monte Carlo);
- codice per la raccolta dei dati su latenza, varianza e successo dei pagamenti;
- esempi di query e parsing degli eventi Nostr (kind:9734 e 9735).

B.2 Note operative

- Alcuni compromessi adottati (es. uso di `NODE_TLS_REJECT_UNAUTHORIZED=0` in ambiente locale) sono accettabili solo in fase di test, non in produzione.
- Per garantire la compatibilità dei client, è consigliato verificare sempre il supporto a NIP-07, NIP-26 e NIP-57.
- I relay usati in test possono avere policy diverse; per scenari reali, è opportuno selezionare relay stabili e ad alta disponibilità.

Appendice C – Formati dei messaggi Nostr

C.1 Modello di un evento Nostr

- **id**: hash SHA-256 dei campi canonici dell'evento;
- **pubkey**: chiave pubblica dell'autore;
- **created_at**: timestamp Unix di creazione;
- **kind**: numero che indica il tipo di evento, ad esempio 0 = metadati utente, 3 = lista seguiti, 10002 = configurazione relay;
- **tags**: lista di array con informazioni aggiuntive (es. **e** = riferimento a un altro evento, **p** = riferimento a un utente);
- **content**: contenuto del messaggio;
- **sig**: firma digitale dell'evento.

C.2 Messaggi dal client al relay

- `["EVENT", <evento JSON>]`: invio di un evento.
- `["REQ", <subscription_id>, <filters...>]`: richiesta di eventi in base a filtri.
- `["CLOSE", <subscription_id>]`: chiusura di una subscription.

C.3 Messaggi dal relay al client

- `["EVENT", <evento JSON>]`: trasmissione di eventi richiesti.
- `["OK", <event_id>, <true|false>, <message>]`: conferma o rifiuto di un evento.
- `["EOSE", <subscription_id>]`: fine degli eventi memorizzati.

- ["CLOSED", <subscription_id>, <message>]: chiusura di una subscription.
- ["NOTICE", <message>]: notifiche o avvisi.

C.4 Autenticazione (NIP-42)

Alcuni relay richiedono autenticazione per accedere a risorse specifiche. Il flusso prevede:

- Il relay invia: ["AUTH", <challenge-string>]
- Il client risponde con un evento di **kind** 22242, firmato, che include la challenge e l'URL del relay.
- Il relay verifica la firma e risponde con ["OK", ...] oppure nega l'accesso.